Table of Contents

Enterprise Knowledge Operations Assistant

(EKOA)

Enterprise Software Architecture Specification (ESAS)

Volume I – Enterprise Software Architecture

Version 1.0

Author: Gamal Gad

July 16, 2026

This specification defines the business, functional, and technical architecture of EKOA — an AI-first enterprise platform combining Retrieval-Augmented Generation (RAG), multi-agent orchestration with LangGraph, the Model Context Protocol (MCP), and enterprise-grade security, integration, and observability capabilities.

Document Information

Field	Value
Document Title	Enterprise Knowledge Operations Assistant (EKOA) — Enterprise Software Architecture Specification
Volume	Volume I — Enterprise Software Architecture
Version	1.0
Author	Gamal Gad
Status	Draft — Core Specification (Chapters 2–21)
Prepared With	Assembled and formatted by Claude (Anthropic) from source drafting content

A Note on This Document

This document compiles the Enterprise Software Architecture Specification (ESAS) for the EKOA project into a single, continuously formatted specification. It was originally drafted chapter-by-chapter, and this version consolidates that material, restores consistent heading and list structure, and renders tables and architecture diagrams properly for a PDF deliverable.

Two things worth knowing as you use it:

- **Chapter 4 (Non-Functional Requirements)** was outlined but never written out in full in the original source — only a list of topics it was meant to cover (performance, scalability, availability, reliability, security, compliance, and related qualities) exists. A placeholder chapter noting this gap has been included in its proper place so the chapter numbering stays intact; it should be completed before the specification is considered final.
- **Volume II–IV** (Developer Implementation Guide, API & Agent Reference, Deployment & Operations Guide) and the detailed appendices (database schema, API reference, MCP tool specs, sequence diagrams, etc.) were planned but not drafted. Their outline is included as an appendix at the end of this document for future work.

Table of Contents

Table of Contents

Chapter 2: Business Requirements Specification (BRS)

2.1 Introduction

The Business Requirements Specification (BRS) defines the business objectives, functional expectations, operational constraints, and stakeholder needs for the Enterprise Knowledge Operations Assistant (EKOA).

The purpose of this chapter is to establish a shared understanding of why the platform is being built before defining how it will be implemented.

Unlike conventional AI assistants that primarily answer user questions, EKOA is designed as a comprehensive enterprise knowledge operating system. It enables organizations to centralize information, orchestrate AI agents, automate workflows, and securely interact with enterprise systems using natural language.

The business requirements described in this chapter serve as the foundation for every architectural, technical, and implementation decision presented throughout the remainder of this specification.

2.2 Business Context

Modern organizations generate vast amounts of structured and unstructured information every day. This information is distributed across numerous platforms including document management systems, project management tools, communication platforms, cloud storage, databases, and source code repositories.

Employees often face significant challenges locating accurate and up-to-date information due to fragmented knowledge sources and inconsistent documentation practices. As organizations continue to adopt AI technologies, the demand for a unified platform capable of securely retrieving, reasoning over, and acting upon enterprise knowledge has grown substantially.

EKOA addresses these challenges by integrating modern AI technologies into a single enterprise platform capable of supporting knowledge retrieval, workflow automation, intelligent collaboration, and decision support.

2.3 Business Objectives

The primary business objectives of EKOA are outlined below.

BO-01: Centralize Organizational Knowledge

Provide a unified platform where enterprise knowledge from multiple systems can be indexed, searched, and accessed through semantic understanding rather than traditional keyword matching.

BO-02: Enhance Employee Productivity

Reduce the time employees spend searching for information by enabling AI-assisted semantic retrieval and contextual reasoning.

BO-03: Improve Decision Making

Enable employees and managers to make informed decisions by providing accurate, explainable, and citation-backed AI-generated insights.

BO-04: Automate Repetitive Tasks

Leverage AI agents to automate repetitive workflows such as:

- Meeting summarization
- Report generation
- Policy comparison
- Email drafting
- Knowledge extraction
- Document classification
- Workflow approvals

BO-05: Enable Secure AI Adoption

Provide enterprise-grade authentication, authorization, governance, auditing, and compliance mechanisms to ensure responsible AI deployment.

BO-06: Reduce Operational Costs

Decrease manual effort associated with information retrieval, document processing, and repetitive administrative tasks.

BO-07: Support Enterprise Integration

Connect with existing enterprise applications without requiring organizations to replace their current technology stack.

2.4 Business Scope

The project scope is divided into In Scope and Out of Scope activities.

In Scope

The platform shall provide:

- AI-powered enterprise search

- Multi-agent reasoning
- Document ingestion
- Retrieval-Augmented Generation (RAG)
- LangGraph orchestration
- LangChain pipelines
- MCP Client
- MCP Server
- Enterprise integrations
- Workflow automation
- User authentication
- Role-Based Access Control (RBAC)
- Organization management
- Team management
- Audit logging
- Monitoring
- Analytics dashboard
- Document versioning
- AI model management
- Prompt management
- Streaming chat interface
- API access
- SDK support
- Docker deployment
- CI/CD pipeline
- Comprehensive testing
- Observability
- Out of Scope (Version 1.0) The following capabilities are intentionally excluded from the first release:
- Autonomous financial transactions
- Direct modification of enterprise databases without approval
- Automatic deployment of production code
- Medical diagnosis
- Legal decision making
- Fully autonomous business operations without human oversight
- These features may be considered in future versions after appropriate governance mechanisms are established.

2.5 Stakeholder Analysis

The platform serves multiple stakeholder groups with distinct responsibilities and expectations.

Stakeholder	Primary Responsibilities
Organization Owner	Creates organizations, manages subscriptions, assigns administrators, defines enterprise policies

Stakeholder	Primary Responsibilities
Administrator	Manages users, permissions, integrations, AI models, audit logs, security settings
Team Manager	Oversees projects, reviews AI-generated reports, approves workflows, monitors team performance
Employee	Searches knowledge, uploads documents, collaborates with AI agents, automates tasks
Guest	Limited access to demonstration workspaces and public knowledge

2.6 User Personas

Organization Owner

The Organization Owner is responsible for configuring and governing the enterprise environment. This user establishes organizational settings, manages billing, defines AI policies, and oversees system-wide operations.

Typical goals include:

- Configure enterprise settings
- Manage subscriptions
- Define organizational policies
- Monitor usage analytics
- Ensure compliance

Administrator Administrators maintain the operational health of the platform by managing users, integrations, permissions, and AI resources.

Typical activities include:

- Managing users
- Configuring AI providers
- Connecting external systems
- Reviewing audit logs
- Managing document repositories
- Monitoring system health
- Team Manager Managers coordinate team activities and ensure AI-generated outputs align with business objectives.
- Common tasks include:
- Reviewing reports
- Approving automated workflows
- Monitoring project progress
- Assigning knowledge repositories
- Managing team permissions

- Employee Employees interact with the platform daily to retrieve knowledge, collaborate with AI agents, and automate routine tasks.
- Typical workflows include:
- Asking questions
- Uploading documents
- Summarizing meetings
- Comparing policies
- Drafting emails
- Generating reports
- Running AI workflows
- Guest Guests have restricted access to demonstration environments for evaluation purposes.
- Capabilities include:
- Viewing public knowledge
- Interacting with demo AI assistants
- Exploring limited platform functionality

2.7 Business Capabilities

The platform shall support the following high-level capabilities:

- Knowledge Management Intelligent document ingestion
- OCR
- Metadata extraction
- Semantic indexing
- Version control
- Knowledge categorization
- AI Assistance Natural language interaction
- Context-aware conversations
- Multi-turn dialogue
- Source citations
- Response explanations
- Workflow Automation AI-driven task orchestration
- Human approval workflows
- Background execution
- Scheduled jobs
- Notification management
- Enterprise Integration GitHub
- Jira
- Slack
- Microsoft Teams
- Google Drive
- OneDrive
- Gmail
- Outlook
- Notion
- Confluence

- REST APIs
- Databases
- Governance Role-based access control
- Audit logging
- Policy enforcement
- AI usage tracking
- Data retention policies

2.8 Business Success Criteria

The success of EKOA will be measured using the following Key Performance Indicators (KPIs):

KPI	Target
Average AI Response Time	< 3 seconds (excluding long-running tasks)
Document Ingestion Success Rate	> 99%
Retrieval Accuracy	> 90%
User Satisfaction Score	> 4.5 / 5
Workflow Automation Success Rate	> 95%
Platform Availability	99.9%
AI Agent Task Completion Rate	> 95%
Search Precision	> 90%
Search Recall	> 85%

2.9 Risks and Assumptions

Risks

- Changes in third-party APIs (e.g., GitHub, Slack, Google Workspace).
- LLM provider rate limits or pricing changes.
- Large document collections affecting retrieval performance.
- Hallucinations from language models if retrieval quality is poor.
- Security risks from misconfigured external integrations.
- Organizational resistance to adopting AI-assisted workflows.
- Assumptions Organizations have permission to index their internal knowledge.
- Users have appropriate access rights to connected systems.
- Internet connectivity is available for cloud-based LLM providers.
- Local deployment with Ollama is available where required.
- The platform will run in a Docker-based environment.

Chapter Summary

This chapter established the business foundation for EKOA by defining its objectives, scope, stakeholders, capabilities, success metrics, and operational assumptions. These requirements will guide every architectural and implementation decision in subsequent chapters.

Chapter 3: Functional Requirements Specification (FRS)

3.1 Introduction

This chapter defines every functional capability the Enterprise Knowledge Operations Assistant (EKOA) shall provide.

Each requirement is uniquely identified to ensure traceability throughout the design, implementation, testing, and deployment phases.

Every requirement in this chapter will later map to:

- UI Screens
- Backend APIs
- Database Tables
- AI Agents
- LangGraph Nodes
- MCP Tools
- Test Cases

3.2 Functional Requirement Classification

The system is divided into twelve functional modules.

Module ID	Module
FR-100	Authentication & Identity
FR-200	Organization Management
FR-300	Workspace Management
FR-400	Knowledge Management
FR-500	AI Assistant
FR-600	Multi-Agent Workflows
FR-700	Enterprise Integrations
FR-800	MCP
FR-900	Administration
FR-1000	Analytics
FR-1100	Notifications
FR-1200	System Settings

Module ID	Module

Each module is described in detail below.

FR-100 Authentication & Identity Purpose Securely identify users while supporting enterprise authentication providers and enforcing organizational security policies.

Features

- FR-101 User Registration The system shall allow users to create accounts using:
- Email
- Google OAuth
- Microsoft OAuth
- GitHub OAuth
- FR-102 Login Users shall authenticate using
- Email + Password
- Google
- Microsoft
- GitHub
- FR-103 JWT Authentication After successful login
- The backend shall generate
- Access Token
- Refresh Token
- using JWT.
- FR-104 Password Reset Users shall reset passwords via secure email verification.
- FR-105 Email Verification Every account must verify ownership before accessing protected resources.
- FR-106 Session Management Users can
- View
- Terminate
- Rename
- their active sessions.
- FR-107 Multi-device Login Support simultaneous login from
- Desktop
- Tablet
- Mobile
- FR-108 MFA (Future Ready) Architecture shall support
- Authenticator Apps
- SMS
- Security Keys
- FR-200 Organization Management Organizations are first-class citizens.
- Every resource belongs to an organization.
- FR-201 Create Organization Users may create organizations.
- Organization includes
- Name
- Logo
- Description

- Brand Colors
- Timezone
- AI Policy
- FR-202 Invite Users Invite via
- Email
- Share Link
- CSV Bulk Upload
- FR-203 Departments Examples
- Engineering
- HR
- Finance
- Marketing
- Legal
- Research
- FR-204 Teams Each department contains teams.
- Each team contains members.
- FR-205 Organization Roles Owner
- Admin
- Manager
- Employee
- Guest
- Custom Roles
- FR-206 AI Policies Administrators define
- Allowed LLMs
- Maximum Tokens
- Daily Budget
- Accessible Tools
- Permitted Integrations
- Sensitive Data Rules
- FR-300 Workspace Management Organizations contain multiple workspaces.
- Examples
- Engineering Workspace
- Marketing Workspace
- Sales Workspace
- Legal Workspace
- Research Workspace
- Workspace features
- Create
- Archive
- Duplicate
- Export
- Import
- Permission Management
- Activity Feed
- Knowledge Collections
- FR-400 Knowledge Management This is one of the largest modules.
- Supported Files PDF
- DOCX

- CSV
- Images
- Audio
- Video
- Markdown
- Text
- Future
- PowerPoint
- Excel
- Upload Methods Drag & Drop
- Cloud Storage
- Git Repository
- REST API
- Email Forwarding
- Bulk Upload
- Folder Sync
- Document Processing Pipeline Every uploaded document goes through:
- 1 Upload
- ↓
- 2 Virus Scan
- ↓
- 3 OCR Detection
- ↓
- 4 Metadata Extraction
- ↓
- 5 Language Detection
- ↓
- 6 Chunking
- ↓
- 7 Embedding Generation
- ↓
- 8 Vector Storage
- ↓
- 9 PostgreSQL Metadata Storage
- ↓
- 10 Ready for Search
- Metadata extracted
- Title
- Author
- Creation Date
- Language
- Keywords
- Department
- Sensitivity Level
- Version
- Source
- Checksum
- Tags

- Version Control
- Each document keeps
- Revision History
- Author History
- Rollback
- Approval State
- FR-500 AI Assistant The AI Assistant is not a chatbot.
- It is an intelligent orchestration layer.
- Capabilities
- Question Answering
- Research
- Summarization
- Document Comparison
- Meeting Notes
- Report Writing
- Email Writing
- Translation
- Policy Analysis
- Contract Review
- SQL Generation
- Code Explanation
- Code Review
- Code Refactoring
- Business Analytics
- Presentation Creation
- Risk Assessment
- Brainstorming
- Workflow Execution
- Every response includes
- Confidence Score
- Sources
- Reasoning Summary
- Related Documents
- Suggested Follow-up Questions
- Supported LLM Providers
- DeepSeek
- Ollama
- Gemini
- Future
- Claude
- OpenAI
- Mistral
- Grok
- Streaming
- Token Streaming
- Citation Streaming
- Live Thinking Status
- Progress Updates

- Conversation Features
- Pinned Chats
- Folders
- Favorites
- Conversation Search
- Conversation Export
- Conversation Sharing
- Conversation Templates
- FR-600 Multi-Agent System Instead of one AI
- We have specialized agents.
- Coordinator Agent
- Planner Agent
- Retriever Agent
- Memory Agent
- Research Agent
- Document Agent
- Workflow Agent
- Analytics Agent
- Security Agent
- Integration Agent
- Code Agent
- Reporting Agent
- Notification Agent
- Quality Assurance Agent
- The Coordinator Agent delegates work.
- Example
- User:

Compare the new HR policy with last year’s version, summarize the changes, draft an email for managers, and create Jira tasks for follow-up.

Coordinator

- ↓
- Planner
- ↓
- Retriever
- ↓
- Document Agent
- ↓
- Research Agent
- ↓
- Reporting Agent
- ↓
- Email Agent
- ↓
- Jira Integration
- ↓
- Completed Result

- Each agent can:
- Call tools
- Query RAG
- Invoke MCP
- Delegate tasks
- Request human approval
- Retry failures
- Log execution

Chapter 3: Functional Requirements Specification (Part II)

FR-700 Enterprise Integrations

7.1 Overview

One of the primary objectives of EKOA is to function as a centralized AI interface capable of interacting with an organization’s existing technology ecosystem.

Rather than requiring organizations to migrate data into a proprietary platform, EKOA connects securely to external systems through APIs, webhooks, and the Model Context Protocol (MCP). This approach enables AI agents to retrieve information, trigger workflows, and perform authorized actions while respecting organizational permissions.

Integrations are grouped into several categories:

- Collaboration Platforms
- Source Control
- Project Management
- Knowledge Bases
- Cloud Storage
- Communication
- Productivity Suites
- Databases
- Custom APIs
- Each integration follows a common lifecycle:
- Authentication
- Permission validation
- Capability discovery
- Tool registration
- Action execution
- Audit logging
- Error handling and retries

7.2 GitHub Integration

The GitHub integration enables AI agents to assist software teams throughout the development lifecycle.

Supported capabilities

- Repository browsing
- Branch management (read-only by default)
- Commit history analysis
- Pull request summarization
- Issue retrieval
- Issue creation (with approval)
- Code search
- README generation
- Documentation updates
- Release note generation
- Example workflow A developer asks:
- “Summarize all open pull requests related to authentication and identify common review comments.”
- The system:
- Uses the GitHub connector to retrieve pull requests.
- The Retrieval Agent gathers metadata.
- The Code Agent reviews diffs.
- The Reporting Agent summarizes recurring themes.
- Results are presented with links back to the relevant pull requests.

7.3 Jira Integration

The Jira connector allows EKOA to interact with project management workflows.

Supported actions include:

- Retrieve issues
- Search by project, sprint, or assignee
- Create issues (approval required)
- Update issue status
- Generate sprint summaries
- Detect blocked tasks
- Suggest backlog prioritization

7.4 Slack and Microsoft Teams

The platform can:

- Search conversations (subject to permissions)
- Summarize discussions
- Draft announcements

- Post notifications
- Create meeting summaries
- Answer questions using channel knowledge
- Sensitive channels are excluded unless explicitly authorized by administrators.

7.5 Notion and Confluence

These integrations provide access to organizational documentation.

Capabilities include:

- Semantic search across pages
- Automatic indexing
- Page summarization
- Cross-document linking
- Documentation quality analysis
- Detection of outdated content

7.6 Google Drive and OneDrive

Supported operations:

- Index folders
- Synchronize changes
- Retrieve documents
- Respect file permissions
- Maintain document version references
- Trigger automatic re-indexing when files change

7.7 Gmail and Outlook

The platform supports:

- Drafting emails
- Summarizing email threads
- Extracting action items
- Categorizing messages
- Scheduling follow-ups
- Human approval before sending

7.8 Custom REST APIs

Organizations may register custom APIs through an Integration Gateway.

Each API definition includes:

- Base URL

- Authentication method
- Available endpoints
- Request/response schemas
- Rate limits
- Timeout policies
- AI agents can invoke these APIs through MCP after authorization.
- FR-800 Model Context Protocol (MCP)

8.1 Overview

The Model Context Protocol enables standardized communication between AI models and external tools.

EKOA implements both sides of the protocol:

- MCP Client
- MCP Server
- This dual implementation allows EKOA to consume external capabilities and expose its own services to other AI systems.

8.2 MCP Client

The MCP Client discovers and interacts with external MCP servers.

Responsibilities:

- Server discovery
- Authentication
- Capability negotiation
- Tool invocation
- Resource retrieval
- Prompt retrieval
- Error handling
- Retry policies
- Supported transports:
- STDIO
- HTTP
- WebSocket (future-ready)

8.3 MCP Server

The MCP Server exposes EKOA functionality to other AI systems.

Exposed capabilities include:

- Knowledge search
- Document retrieval
- Workflow execution

- Organization metadata
- Analytics
- Reporting
- AI-generated summaries
- Authentication is mandatory for all requests.

8.4 Tool Registry

All available tools are registered in a central Tool Registry.

Each tool definition contains:

- Name
- Description
- Version
- Required permissions
- Input schema
- Output schema
- Timeout
- Retry policy
- Audit category
- The registry enables dynamic discovery and simplifies the addition of new integrations.
- FR-900 Administration

9.1 User Administration

Administrators can:

- Create users
- Disable accounts
- Reset passwords
- Assign roles
- Manage organizations
- Review active sessions
- Force logout
- Configure security policies

9.2 AI Provider Management

Administrators define:

- Enabled LLM providers
- API keys
- Usage quotas
- Default models
- Fallback order

- Maximum context size
- Cost limits

9.3 Prompt Management

Reusable prompt templates can be:

- Created
- Versioned
- Tested
- Shared
- Deprecated
- Templates are categorized by use case (e.g., document analysis, report generation, coding assistance).

9.4 Knowledge Source Management

Administrators manage:

- Connected repositories
- Sync schedules
- Retention policies
- Indexing rules
- Access permissions
- Data classification
- FR-1000 Analytics

10.1 Usage Analytics

The Analytics Dashboard provides:

- Daily active users
- AI requests
- Average response time
- Token consumption
- Workflow executions
- Document uploads
- Search success rate

10.2 Model Performance

Metrics include:

- Latency
- Throughput
- Error rate

- Hallucination reports
- Retrieval quality
- User feedback scores
- These metrics help administrators optimize provider selection and prompt design.

10.3 Knowledge Insights

The system identifies:

- Most accessed documents
- Frequently asked questions
- Knowledge gaps
- Duplicate content
- Outdated documentation
- Popular workflows
- FR-1100 Notifications The Notification Service informs users about important events.
- Supported channels:
- In-app notifications
- Email
- Slack
- Microsoft Teams
- Webhooks (future-ready)
- Notification types:
- Workflow completion
- Document processing
- Approval requests
- Integration failures
- Security alerts
- System maintenance
- Users can configure notification preferences individually.
- FR-1200 System Settings System settings are organized into several categories:
- Organization Settings Name
- Branding
- Timezone
- Locale
- AI Settings Default provider
- Allowed providers
- Temperature
- Maximum tokens
- Embedding model
- Reranking model
- Security Settings Password policies
- Session duration
- OAuth providers
- IP restrictions

- Audit retention
- Storage Settings Document retention
- Backup schedules
- Vector database maintenance
- Cache configuration
- Integration Settings Connected services
- API credentials
- Sync intervals
- Webhook configuration

3.10 Error Handling Requirements

The platform shall:

- Return standardized API error responses.
- Distinguish user-facing messages from internal logs.
- Retry transient failures where appropriate.
- Surface actionable guidance to users.
- Log all failures with correlation IDs for tracing.

3.11 Functional Traceability

Every functional requirement defined in this chapter will be mapped to:

- User interface components.
- Backend APIs.
- Database entities.
- AI agents.
- LangGraph nodes.
- MCP tools.
- Test cases.
- Monitoring metrics.
- This traceability ensures that each requirement can be verified during development and testing.

Chapter Summary

This chapter defined the complete functional scope of EKOA, covering authentication, organization management, knowledge management, AI capabilities, multi-agent workflows, enterprise integrations, MCP, administration, analytics, notifications, and system settings. Together, these requirements establish the functional blueprint for the platform and provide the basis for architectural design.

Chapter 4: Non-Functional Requirements (NFR) — Outline Only

> **Status note:** This chapter was scoped during drafting but its detailed requirements were never written out in the original source material — the project moved on to Chapter 5 before returning to it. The topic list below is what this chapter was intended to cover. It should be expanded into full requirement statements (in the style of the `BO-##` and `FR-###` requirements used in Chapters 2 and 3) before this specification is treated as complete.

4.1 Planned Scope

This chapter is intended to define the qualities that make EKOA enterprise-ready, including:

- Performance targets
- Scalability
- Availability
- Reliability
- Security
- Compliance
- Maintainability
- Observability
- Disaster recovery
- Cost optimization
- Internationalization
- Accessibility
- Sustainability
- Operational excellence

4.2 Suggested Next Step

Each item above should be turned into a numbered non-functional requirement (e.g., `NFR-01: API Response Latency`) with a measurable target, following the same traceability approach used for the functional requirements in Chapter 3 (Section 3.11) — mapping each NFR to the architectural decisions, monitoring metrics, and test cases that verify it. Chapters 15 (DevOps & Deployment), 19 (Observability & AIOps), and

21 (Performance Engineering & Scalability) already contain relevant technical detail that can be cross-referenced when writing these requirements.

Chapter 5: Technology Stack & Architecture Decision Records (ADR)

5.1 Introduction

Technology selection is one of the most critical aspects of enterprise software architecture. Every framework, library, database, and service introduces capabilities, constraints, operational overhead, and long-term maintenance implications.

This chapter documents the major architectural decisions made for EKOA, including the rationale behind each choice, the alternatives considered, and the expected impact on the system.

The guiding principles for technology selection are:

- Open standards where practical.
- Modular and replaceable components.
- Strong developer ecosystem.
- Production readiness.
- Enterprise scalability.
- Long-term maintainability.
- Provider independence.
- Community adoption and documentation.
- ADR-001 — Frontend Framework Decision Use Next.js 15 with TypeScript as the primary frontend framework.
- Alternatives Considered React + Vite
- Angular
- Vue.js
- SvelteKit
- Remix

Rationale Next.js provides a mature React ecosystem with support for server-side rendering, server components, routing, API integration, and production optimizations. It is widely adopted for enterprise web applications and offers excellent performance and developer experience.

TypeScript improves maintainability, reduces runtime errors, and enhances tooling.

Consequences

- Positive Excellent performance

- Strong ecosystem
- Scalable project structure
- Enterprise adoption
- Type safety
- Negative Slightly steeper learning curve
- Larger framework surface area compared to Vite
- ADR-002 — UI Framework Decision Use Tailwind CSS with shadcn/ui.
- Alternatives Material UI
- Ant Design
- Chakra UI
- Bootstrap

Rationale Tailwind CSS provides utility-first styling that is highly customizable. shadcn/ui offers accessible, composable components without locking the project into a rigid design system.

This combination supports modern enterprise dashboards while remaining flexible for future branding.

ADR-003 — Backend Framework Decision Use FastAPI.

Alternatives

- Django
- Flask
- Express.js
- NestJS
- Spring Boot
- Rationale FastAPI is well suited for AI-driven systems due to:
- High performance
- Native async support
- Automatic OpenAPI generation
- Pydantic validation
- Excellent Python ecosystem integration
- Since most AI tooling is Python-based, FastAPI minimizes integration complexity.
- ADR-004 — Database Decision Use PostgreSQL as the primary relational database.
- Alternatives MySQL
- MongoDB
- SQL Server
- Rationale PostgreSQL provides:
- ACID compliance
- Rich indexing
- JSON support
- Mature tooling
- Strong community
- Excellent support for enterprise workloads
- Structured organizational data (users, permissions, audit logs, workflows) fits naturally into a relational model.
- ADR-005 — Vector Database Decision Use Qdrant.

- Alternatives Pinecone
- Chroma
- Weaviate
- Milvus
- pgvector
- Rationale Qdrant offers:
- High-performance vector search
- Rich metadata filtering
- Open-source deployment
- Docker compatibility
- Production readiness
- Horizontal scalability
- It integrates well with LangChain and supports hybrid retrieval patterns.
- ADR-006 — Cache and Message Broker Decision Use Redis.
- Alternatives Memcached
- RabbitMQ (for caching use cases)
- Hazelcast
- Rationale Redis supports:
- Distributed caching
- Session storage
- Background job queues
- Rate limiting
- Pub/Sub messaging
- Its versatility reduces infrastructure complexity.
- ADR-007 — AI Framework Decision Use LangChain.
- Alternatives LlamaIndex
- Haystack
- Custom orchestration
- Rationale LangChain provides:
- Standardized abstractions for LLM interaction
- Tool calling
- Prompt templates
- Retrievers
- Output parsers
- Memory components
- It forms the foundation for higher-level orchestration with LangGraph.
- ADR-008 — Multi-Agent Orchestration Decision Use LangGraph.
- Alternatives CrewAI
- AutoGen
- Custom state machine

Rationale LangGraph introduces graph-based orchestration with explicit state management, conditional routing, parallel execution, and durable workflows. These capabilities are essential for complex enterprise processes involving multiple specialized agents.

ADR-009 — LLM Strategy Decision Implement a provider-agnostic LLM abstraction layer.

Initial Providers

- DeepSeek (cloud)
- Ollama (local)
- Google Gemini (free tier)
- Future Providers OpenAI
- Claude
- Mistral
- Grok
- Azure OpenAI

Rationale Abstracting model providers prevents vendor lock-in and allows organizations to choose models based on cost, privacy, or performance. New providers can be added through adapter implementations without changing business logic.

ADR-010 — Model Context Protocol (MCP) Decision Implement both an MCP Client and an MCP Server.

Alternatives

- REST-only integrations
- Custom tool interfaces

Rationale MCP is emerging as a standard for AI tool interoperability. Supporting both client and server roles allows EKOA to consume external capabilities and expose its own services to other AI systems.

ADR-011 — Deployment Decision Use Docker Compose for development and design the architecture to be Kubernetes-ready.

Alternatives

- Native installations
- Docker only
- Direct Kubernetes from day one

Rationale Docker Compose simplifies local development while keeping service boundaries aligned with future container orchestration platforms.

ADR-012 — CI/CD Decision Use GitHub Actions.

Pipeline Stages

- Linting
- Unit testing
- Integration testing
- Security scanning
- Docker image build
- Artifact publishing
- Deployment (future)
- Rationale GitHub Actions integrates naturally with the repository and supports automated quality gates.
- ADR-013 — Observability Decision Adopt a comprehensive observability stack:
- OpenTelemetry

- Prometheus
- Grafana
- LangSmith
- Sentry
- Rationale Each tool addresses a different aspect of system health:
- OpenTelemetry: distributed tracing
- Prometheus: metrics collection
- Grafana: dashboards
- LangSmith: AI workflow debugging
- Sentry: error tracking
- Together they provide full-stack visibility.
- ADR-014 — Security Decision Security will follow a Zero Trust philosophy.
- Core principles:
- Least privilege
- Explicit authentication
- Continuous authorization
- Encryption in transit and at rest
- Audit logging
- Secret management
- Secure defaults
- ADR-015 — Architectural Style Decision Adopt a modular microservice architecture.
- Core Services Frontend
- API Gateway
- Authentication Service
- User Service
- Organization Service
- AI Service
- RAG Service
- Document Service
- Workflow Service
- Integration Service
- MCP Service
- Notification Service
- Analytics Service
- Each service owns its data and communicates through well-defined APIs or asynchronous messaging where appropriate.

5.2 Technology Summary

Layer	Technology
Frontend	Next.js 15 + TypeScript
UI	Tailwind CSS + shadcn/ui
Backend	FastAPI
ORM	SQLAlchemy

Layer	Technology
Migrations	Alembic
Database	PostgreSQL
Vector Database	Qdrant
Cache	Redis
AI Framework	LangChain
Agent Orchestration	LangGraph
LLM Providers	DeepSeek, Ollama, Gemini
Authentication	JWT + OAuth
Deployment	Docker Compose
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana
Tracing	OpenTelemetry
AI Observability	LangSmith
Error Tracking	Sentry

Chapter Summary

This chapter established the architectural technology stack and documented the rationale behind each major technology decision. By using an Architecture Decision Record (ADR) approach, EKOA ensures that technology choices remain transparent, justifiable, and maintainable as the platform evolves.

Chapter 6: High-Level System Architecture

6.1 Introduction

The Enterprise Knowledge Operations Assistant (EKOA) is designed as a modular, cloud-native, AI-first enterprise platform. Its architecture emphasizes scalability, maintainability, security, extensibility, and interoperability with enterprise ecosystems.

Rather than relying on a monolithic application, EKOA adopts a modular service-oriented architecture. Each major business capability is encapsulated within its own service boundary, allowing independent development, testing, deployment, and scaling.

The system follows an API-first philosophy, exposing well-defined interfaces between services and enabling future expansion to mobile clients, desktop applications, and third-party integrations.

6.2 Architectural Principles

The architecture is guided by the following principles:

1. Separation of Concerns

Each service is responsible for a single domain, such as authentication, document management, or AI orchestration.

2. Loose Coupling

Services communicate through stable APIs and asynchronous messaging where appropriate, minimizing dependencies.

3. High Cohesion

Related functionality is grouped together to improve maintainability and clarity.

4. AI-First Design

Artificial intelligence is treated as a core platform capability rather than an add-on feature.

5. Provider Independence

The platform abstracts LLM providers, embedding models, and integrations to avoid vendor lock-in.

6. Security by Design

Authentication, authorization, encryption, and auditing are embedded into every layer of the architecture.

7. Observability

Every service emits metrics, logs, and traces to support monitoring, debugging, and optimization.

8. Extensibility

New AI agents, integrations, and services can be introduced with minimal changes to existing components.

6.3 Architectural Overview

At a high level, EKOA consists of the following logical layers:

- Presentation Layer
- API Gateway Layer
- Application Services Layer
- AI Intelligence Layer
- Integration Layer
- Data Layer
- Infrastructure Layer
- Each layer has distinct responsibilities and communicates through clearly defined interfaces.

6.4 Presentation Layer

The Presentation Layer provides the user interface for interacting with EKOA.

Components

- Web Application (Next.js)
- Admin Dashboard
- Organization Portal

- Workspace Dashboard
- AI Chat Interface
- Knowledge Explorer
- Workflow Builder
- Analytics Dashboard
- Settings Portal
- Responsibilities User authentication
- Data visualization
- Streaming AI responses
- File uploads
- Workflow management
- User preferences
- Accessibility
- Responsive design

6.5 API Gateway Layer

The API Gateway acts as the single entry point for all client requests.

Responsibilities

- Request routing
- Authentication validation
- Rate limiting
- Request logging
- API versioning
- Response compression
- CORS management
- Request tracing
- Benefits Centralized security
- Simplified client interactions
- Unified API surface
- Easier monitoring

6.6 Application Services Layer

This layer contains the core business logic.

1. Authentication Service

Responsibilities:

- JWT issuance
- OAuth login
- Session management
- Role validation

2. Organization Service

Responsibilities:

- Organizations
- Departments
- Teams
- Membership
- Permissions

3. User Service

Responsibilities:

- Profiles
- Preferences
- Activity history
- Notification settings

4. Document Service

Responsibilities:

- File uploads
- Metadata extraction
- Versioning
- Storage
- OCR coordination

5. Workflow Service

Responsibilities:

- Workflow definitions
- Workflow execution
- Scheduling
- Approval management

6. Notification Service

Responsibilities:

- Email
- In-app notifications
- Slack
- Teams
- Webhooks

7. Analytics Service

Responsibilities:

- Usage metrics
- AI statistics
- Knowledge insights
- Operational dashboards

6.7 AI Intelligence Layer

This is the distinguishing layer of EKOA and contains the platform's AI capabilities.

Components

- AI Orchestrator
- LangGraph Runtime
- LangChain Runtime
- Prompt Manager
- Memory Manager
- LLM Adapter
- Embedding Service
- RAG Engine
- Agent Registry
- Responsibilities AI reasoning
- Agent orchestration
- Prompt execution
- Tool calling
- Context management
- Retrieval
- Memory
- Model selection

6.8 Integration Layer

The Integration Layer enables communication with external systems.

Integration Gateway

- The gateway provides a unified interface for all enterprise connectors.
- Supported Integrations GitHub
- Jira
- Slack
- Microsoft Teams
- Notion
- Confluence
- Google Drive
- OneDrive
- Gmail

- Outlook
- Google Calendar
- REST APIs
- Databases
- Responsibilities Authentication
- API communication
- Data transformation
- Retry handling
- Rate limiting
- Audit logging

6.9 MCP Layer

The MCP Layer enables standardized AI interoperability.

MCP Client

Responsibilities:

- Discover external MCP servers
- Invoke tools
- Retrieve resources
- Manage sessions
- MCP Server Responsibilities:
- Expose internal capabilities
- Register tools
- Handle requests
- Enforce permissions
- This dual implementation ensures EKOA can both consume and provide AI capabilities.

6.10 Data Layer

The Data Layer manages persistent and transient data.

PostgreSQL

Stores:

- Users
- Organizations
- Teams
- Documents
- Metadata
- Workflows
- Audit logs
- Notifications
- Configuration
- Qdrant Stores:

- Document embeddings
- Semantic indexes
- Metadata filters
- Redis Stores:
- Sessions
- Caches
- Background queues
- Rate limiting counters
- Object Storage Stores:
- Uploaded documents
- Images
- Audio
- Video
- Processed artifacts

6.11 Infrastructure Layer

The Infrastructure Layer provides runtime services.

Components

- Docker Compose
- Reverse Proxy
- Monitoring
- Logging
- Tracing
- Secrets Management
- CI/CD

6.12 Request Lifecycle

Example: User asks,

“Summarize the latest HR policy changes and create a Jira task for managers.”

User submits request through the web application.

API Gateway authenticates the request.

AI Orchestrator receives the query.

LangGraph creates an execution plan.

Retrieval Agent queries the RAG Engine.

Document Agent extracts relevant policy information.

Reporting Agent prepares the summary.

Integration Agent prepares the Jira task.

If required, the Workflow Service requests manager approval.

The Integration Gateway creates the Jira issue.

AI Orchestrator compiles the final response.

The response is streamed back to the user with citations and status updates.

6.13 Communication Patterns

Synchronous

Used for:

- Authentication
- Dashboard data
- Search queries
- Interactive chat
- Asynchronous Used for:
- Document ingestion
- Embedding generation
- OCR
- Audio transcription
- Video analysis
- Long-running workflows
- Notification delivery
- This separation ensures that heavy workloads do not impact user responsiveness.

6.14 Fault Tolerance

The architecture includes:

- Health checks
- Automatic retries
- Circuit breakers
- Graceful degradation
- Dead-letter queues (for background jobs)
- Timeouts
- Idempotent workflow execution

6.15 Security Boundaries

Security is enforced at multiple layers:

- API Gateway authentication
- Service-to-service authorization
- Organization-level isolation
- Role-based permissions
- Encrypted communications

- Secret management
- Audit logging
- No service may bypass authorization checks, even when communicating internally.

6.16 Deployment Topology

A standard deployment consists of:

- Next.js frontend
- API Gateway
- FastAPI backend services
- PostgreSQL
- Qdrant
- Redis
- Object Storage
- Background Worker
- MCP Service
- Monitoring stack
- Reverse Proxy
- Each service runs in its own Docker container and can later be migrated to Kubernetes without architectural changes.

6.17 Architecture Benefits

This architecture provides:

- Independent service scaling
- Easier maintenance
- Clear domain boundaries
- Flexible AI provider integration
- Enterprise-grade security
- High availability
- Future extensibility
- Improved developer productivity

Chapter Summary

This chapter established the overall architectural blueprint of EKOA, defining the system's layered design, service boundaries, communication patterns, data management, and deployment topology. It serves as the foundation upon which all subsequent detailed designs—including databases, APIs, RAG, LangGraph, and MCP—will be built.

Chapter 7: Repository Structure & Modular Monorepo Design

7.1 Introduction

The EKOA codebase shall be organized as a modular monorepo, where all application components are maintained in a single repository while preserving clear boundaries between services and shared libraries.

This approach provides:

- Centralized version control
- Simplified dependency management
- Shared TypeScript and Python models
- Consistent coding standards
- Easier onboarding
- Unified CI/CD pipelines
- Atomic commits across multiple services
- Although developed in a monorepo, each service remains independently deployable.

7.2 Repository Overview

```
ekoa/
│
├── apps/
│   ├── web/
│   ├── api/
│   ├── ai/
│   ├── worker/
│   ├── gateway/
│   ├── mcp-server/
│   └── admin/
│
├── packages/
│   ├── shared-types/
│   ├── shared-config/
│   ├── shared-prompts/
```

```
│   ├── shared-utils/
│   ├── ui-components/
│   └── sdk/
│
├── infrastructure/
│   ├── docker/
│   ├── monitoring/
│   ├── nginx/
│   ├── scripts/
│   └── github-actions/
│
├── docs/
├── tests/
├── examples/
└── README.md
```

The repository is divided into Applications, Shared Packages, Infrastructure, and Documentation.

7.3 Applications Directory

/apps/web Primary user-facing application.

Technology:

- Next.js 15
- TypeScript
- Tailwind CSS
- shadcn/ui
- Responsibilities:
- Authentication
- AI Chat
- Dashboard
- Knowledge Search
- Workflow Builder
- Document Upload
- Analytics
- Organization Management
- /apps/admin Dedicated administration portal.
- Features:
- User Management
- Organization Management
- AI Provider Configuration
- Integration Management
- Audit Logs
- Security Policies
- Monitoring Dashboard
- /apps/api Main FastAPI application.

- Responsibilities:
- REST APIs
- Authentication
- RBAC
- Business Logic
- Request Validation
- OpenAPI Documentation
- /apps/ai Dedicated AI service.
- Contains:
- LangGraph Runtime
- LangChain
- Prompt Manager
- Memory Manager
- Model Router
- Tool Executor
- Agent Registry
- RAG Orchestrator
- This service is isolated so that AI workloads can scale independently.
- /apps/worker Background processing service.
- Handles:
- OCR
- Audio transcription
- Video processing
- Embedding generation
- Large document ingestion
- Scheduled workflows
- Notifications
- This service communicates through Redis-backed job queues.
- /apps/gateway API Gateway.
- Responsibilities:
- Request routing
- Authentication validation
- Rate limiting
- API versioning
- Request logging
- Correlation IDs
- /apps/mcp-server Implements the Model Context Protocol server.
- Responsibilities:
- Tool registration
- Resource exposure
- Prompt exposure
- Session handling
- Authentication
- Capability discovery

7.4 Shared Packages

/packages/shared-types Contains shared data models.

Examples:

- User Organization Workspace Document ChatMessage Workflow AgentTask Shared between frontend and backend to reduce duplication.
- /packages/shared-config Centralized configuration.
- Includes:
- Environment parsing
- Feature flags
- Default AI settings
- Logging configuration
- Retry policies
- /packages/shared-prompts Version-controlled prompt templates.
- Examples:
- summarize_document.md
- compare_documents.md
- draft_email.md
- research_agent.md
- code_review.md
- analytics.md Every prompt is versioned and testable.
- /packages/shared-utils Common helper functions.
- Examples:
- Date formatting
- Validation
- Error handling
- File utilities
- Retry helpers
- Token counting
- /packages/ui-components Reusable frontend components.
- Examples:
- Button
- Dialog
- Table
- Sidebar
- Navbar
- AIChat
- FileUploader
- KnowledgeCard /packages/sdk Official SDK for external developers.
- Supports:
- JavaScript
- Python
- Future:
- Java
- Go
- C#

7.5 Infrastructure Directory

Docker

Contains:

- Dockerfiles
- Compose files
- Development containers
- Monitoring Contains configuration for:
- Prometheus
- Grafana
- OpenTelemetry
- LangSmith
- Sentry
- NGINX Reverse proxy configuration.
- Responsibilities:
- HTTPS termination
- Compression
- Routing
- Security headers
- GitHub Actions Pipeline definitions.
- Examples:
- lint.yml
- tests.yml
- docker.yml
- security.yml
- release.yml

7.6 Documentation

The /docs directory contains:

- Architecture
- API
- Database
- Agents
- RAG
- Deployment
- Security
- ADR
- User Guide
- Developer Guide Documentation is treated as a first-class part of the project.

7.7 Testing Structure

tests/

unit/

integration/

e2e/

performance/

rag/

agents/

security/ Each category targets a different level of system verification.

7.8 Configuration Management

Environment variables are organized by service.

Example:

- .env.web
- .env.api
- .env.ai
- .env.worker
- .env.gateway
- .env.monitoring Secrets are never committed to version control and are managed through environment injection.

7.9 Dependency Rules

To maintain modularity:

- Applications may depend on shared packages.
- Shared packages must not depend on applications.
- AI service communicates through APIs or message queues.
- Circular dependencies are prohibited.
- These rules ensure clean architectural boundaries.

7.10 Naming Conventions

Services

- auth-service
- document-service
- ai-service
- APIs /api/v1/chat
- /api/v1/documents
- /api/v1/workflows
- Database Tables users
- organizations

- documents
- workflows
- Variables camelCase for JavaScript/TypeScript.
- snake_case for Python variables where appropriate.
- PascalCase for classes.
- Consistent naming improves readability and maintainability.

7.11 Development Workflow

Create feature branch.

Implement changes.

Run linting.

Execute unit tests.

Run integration tests.

Open pull request.

Automated CI validation.

Code review.

Merge into main.

Deploy through CI/CD.

Chapter Summary

This chapter defined the physical organization of the EKOA codebase, including the modular monorepo layout, application boundaries, shared packages, infrastructure configuration, testing strategy, and development workflow. This structure enables efficient collaboration while preserving clean architectural separation and future scalability.

Chapter 8: Database Architecture & Data Modeling

8.1 Introduction

The database architecture provides the persistent foundation of EKOA. It is responsible for storing structured business data, semantic knowledge metadata, AI execution history, organizational configurations, and system state.

The design follows these principles:

- Normalize business data.
- Support multi-tenancy.
- Ensure ACID compliance.
- Enable horizontal scaling where appropriate.
- Maintain auditability.
- Optimize for read-heavy AI workloads.
- Separate transactional data from vector embeddings.
- The data layer is composed of:
- PostgreSQL
- Qdrant
- Redis
- Object Storage
- Each technology is selected based on its strengths and responsibilities.

8.2 Database Overview

Component	Purpose
PostgreSQL	Transactional business data
Qdrant	Vector embeddings and semantic search
Redis	Caching, sessions, queues
Object Storage	Files and media

These systems work together while maintaining clear boundaries.

8.3 Multi-Tenant Strategy

Every organization is logically isolated.

Core concepts:

- Organization
- Department
- Team
- Workspace
- User
- Each business object belongs to an organization.

```
Organization
    |
Departments
    |
Teams
    |
Workspaces
    |
Resources
```

This hierarchy supports permission inheritance and simplifies access control.

8.4 Core Entities

The following entities form the core of the relational model.

Identity & Access

- users
- roles
- permissions
- organizations
- organization_members
- departments
- teams
- team_members
- api_keys
- sessions
- Knowledge Management documents
- document_versions
- document_tags
- document_chunks
- knowledge_collections
- collection_documents
- uploads
- processing_jobs
- AI conversations

- conversation_messages
- prompts
- prompt_versions
- ai_models
- embeddings
- agent_runs
- workflows
- workflow_runs
- workflow_steps
- Integrations integrations
- integration_tokens
- webhooks
- external_resources
- Monitoring audit_logs
- activity_logs
- notifications
- metrics
- system_events

8.5 Entity Relationships

At a high level:

```
Organization
    ├── Users
    ├── Departments
    ├── Teams
    ├── Workspaces
    ├── Documents
    ├── Integrations
    ├── Workflows
    └── Audit Logs
Relationships include:
```

One Organization → Many Users

One User → Many Sessions

One Workspace → Many Documents

One Document → Many Versions

One Conversation → Many Messages

One Workflow → Many Executions

8.6 Users Table

The users table stores user identity information.

Key fields:

- id (UUID)
- organization_id
- email
- full_name
- avatar_url
- status
- locale
- timezone
- created_at
- updated_at
- Indexes:
- email (unique)
- organization_id
- status

8.7 Organizations Table

Stores tenant information.

Fields include:

- id
- name
- slug
- logo_url
- plan
- ai_policy_id
- created_at
- This table is the root of the multi-tenant hierarchy.

8.8 Documents Table

Represents uploaded knowledge assets.

Fields:

- id
- organization_id
- workspace_id
- owner_id
- title
- file_type
- storage_path
- language
- checksum
- sensitivity_level

- current_version
- created_at
- Documents never store embeddings directly.

8.9 Document Versions

Every document version is immutable.

Each version stores:

- version_number
- uploaded_by
- created_at
- processing_status
- change_summary
- This enables rollback and historical analysis.

8.10 Document Chunks

Each processed document is divided into semantic chunks.

Fields:

- chunk_id
- document_id
- version_id
- chunk_index
- text
- token_count
- embedding_id
- Chunk metadata includes:
- page number
- section title
- timestamps (audio/video)
- confidence score

8.11 Conversations

Stores AI chat sessions.

Relationships:

- Conversation
- ↓
- Messages
- ↓
- Tool Calls
- ↓

- Agent Runs
- Each conversation maintains context independently.

8.12 Agent Runs

Every AI task is recorded.

Fields:

- agent_name
- model
- prompt_version
- start_time
- end_time
- status
- token_usage
- cost_estimate
- parent_agent
- This provides traceability and debugging support.

8.13 Workflows

Workflow definitions include:

- name
- description
- graph_definition
- trigger
- approval_required
- schedule
- Workflow executions are stored separately for auditing.

8.14 Audit Logs

Every sensitive action is logged.

Examples:

- Login
- Role change
- Document deletion
- Integration added
- Workflow approved
- API key created
- Each entry includes:
- actor
- action
- resource

- timestamp
- IP address
- correlation ID

8.15 Soft Deletes

Business records are never immediately removed.

Instead, they include:

- deleted_at
- deleted_by
- deletion_reason
- This supports recovery and auditing.

8.16 Indexing Strategy

Indexes are created on:

- foreign keys
- email
- organization_id
- workspace_id
- created_at
- status
- Composite indexes are added for common query patterns, such as (organization_id, created_at).

8.17 Qdrant Collections

Embeddings are stored separately from relational data.

Proposed collections:

- document_embeddings
- conversation_memory
- prompt_examples
- workflow_context
- Payload metadata includes:
- organization_id
- document_id
- workspace_id
- language
- permissions
- tags
- This allows efficient filtering during retrieval.

8.18 Redis Usage

Redis supports:

- User sessions
- JWT blacklists
- Rate limiting
- Caching
- Background job queues
- Temporary conversation state
- Key naming convention:
- session:{user_id} cache:search:{query_hash} queue:embedding workflow:{workflow_id} TTL policies vary by data type.

8.19 Data Lifecycle

A document follows this lifecycle:

- Upload
- Virus scan
- Metadata extraction
- OCR (if required)
- Chunking
- Embedding generation
- Vector indexing
- PostgreSQL metadata update
- Search availability
- Conversation data, workflow history, and audit logs follow similar lifecycle management with configurable retention policies.

8.20 Backup Strategy

PostgreSQL: nightly full backups + WAL archiving.

Qdrant: scheduled snapshots.

Redis: persistence enabled for critical data.

Object Storage: versioning and replication.

Regular recovery drills should validate backup integrity.

Chapter Summary

This chapter defined the data architecture of EKOA, including the hybrid multi-tenant model, core relational entities, vector storage, caching strategy, indexing, lifecycle management, and backup approach. These designs provide a robust foundation for the platform's APIs, AI workflows, and enterprise operations.

Chapter 9: API Architecture & Service Contracts

9.1 Introduction

The EKOA API provides a standardized interface for all platform interactions. It follows an API-first approach, ensuring that every capability is accessible through documented, versioned, and secure interfaces.

The API is designed to support:

- Web frontend
- Future mobile applications
- Desktop clients
- External enterprise systems
- MCP integrations
- AI services
- Administrative tools
- All APIs are documented using the OpenAPI specification and follow consistent naming, validation, and error-handling conventions.

9.2 API Design Principles

The API adheres to the following principles:

- Resource-oriented design.
- Stateless requests where applicable.
- Predictable URL structures.
- Consistent request and response formats.
- Versioned endpoints.
- Strong validation.
- Idempotent operations where appropriate.
- Clear error reporting.
- Secure by default.

9.3 API Base Structure

Base URL:

- /api/v1/ Examples:

/api/v1/auth/login /api/v1/users /api/v1/documents /api/v1/chat /api/v1/workflows /api/v1/integrations Future versions (/api/v2) can coexist without breaking existing clients.

9.4 Authentication

Protected endpoints require a valid JWT access token.

Example header:

- Authorization: Bearer Authentication flow:
- User logs in.
- Access token and refresh token are issued.
- Access token is sent with each request.
- Refresh endpoint renews expired tokens.
- Logout invalidates the session.

9.5 Common Request Format

Example JSON payload:

{ “workspaceId”: “uuid”, “query”: “Summarize the latest HR policy”, “options”: { “stream”: true, “includeSources”: true } } All request bodies are validated using Pydantic models.

9.6 Standard Response Format

Successful responses follow a consistent structure:

- { “success”: true, “data”: {}, “metadata”: { “requestId”: “uuid”, “timestamp”: “2026-07-15T12:00:00Z” } }

9.7 Error Response Format

Errors include machine-readable and human-readable information:

{ “success”: false, “error”: { “code”: “DOCUMENT_NOT_FOUND”, “message”: “The requested document could not be found.”, “details”: {} }, “metadata”: { “requestId”: “uuid” } } This format simplifies debugging and client-side error handling.

9.8 Pagination

List endpoints support pagination:

- Query parameters:

- ?page=1 &pageSize=25 Response metadata:
- { “page”: 1, “pageSize”: 25, “totalItems”: 143, “totalPages”: 6 }

9.9 Filtering and Sorting

Standard query parameters:

- ?status=active &workspaceId=… &sortBy=createdAt &order=desc Filtering rules are documented per endpoint.

9.10 REST Endpoint Groups

Authentication

- POST /auth/register
- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- POST /auth/forgot-password
- POST /auth/reset-password
- Users GET /users/me
- PATCH /users/me
- GET /users
- GET /users/{id}
- PATCH /users/{id}
- DELETE /users/{id}
- Organizations GET /organizations
- POST /organizations
- GET /organizations/{id}
- PATCH /organizations/{id}
- DELETE /organizations/{id}
- Workspaces GET /workspaces
- POST /workspaces
- PATCH /workspaces/{id}
- DELETE /workspaces/{id}
- Documents POST /documents/upload
- GET /documents
- GET /documents/{id}
- PATCH /documents/{id}
- DELETE /documents/{id}
- POST /documents/{id}/reindex
- AI Chat POST /chat
- POST /chat/stream
- GET /conversations
- GET /conversations/{id}
- DELETE /conversations/{id}
- Workflows GET /workflows

- POST /workflows
- POST /workflows/{id}/execute
- PATCH /workflows/{id}
- DELETE /workflows/{id}
- Integrations GET /integrations
- POST /integrations
- PATCH /integrations/{id}
- DELETE /integrations/{id}
- POST /integrations/{id}/sync

9.11 File Upload API

Supported content:

- PDF
- DOCX
- CSV
- Images
- Audio
- Video
- Upload flow:
- Client requests upload.
- Server validates metadata.
- File stored in object storage.
- Processing job created.
- Worker begins ingestion.
- Client receives job status updates.

9.12 Streaming APIs

AI responses are streamed over WebSockets (or SSE where configured).

Streaming events:

connection_open thinking retrieval_started retrieval_completed tool_call token citation workflow_update completed error This allows the UI to display progress in real time.

9.13 WebSocket Channels

Example channels:

- /ws/chat /ws/workflows /ws/notifications Messages include a correlation ID to associate updates with the originating request.

9.14 Idempotency

Endpoints that create resources may accept an Idempotency-Key header.

This prevents duplicate operations caused by retries or network failures.

Example use cases:

- Document upload
- Workflow execution
- Integration creation

9.15 Rate Limiting

Rate limits are enforced per user and API key.

Example defaults:

Endpoint	Limit
Authentication	10 requests/minute
Chat	60 requests/minute
Document Upload	20 requests/minute
Search	120 requests/minute

Organizations can configure custom limits based on subscription plans.

9.16 API Security

Security measures include:

- JWT authentication.
- RBAC authorization.
- Input validation.
- Request size limits.
- CSRF protection (where applicable).
- Secure HTTP headers.
- Audit logging.
- IP allow/block lists (optional).

9.17 Service-to-Service Communication

Internal services communicate using authenticated HTTP APIs for synchronous requests and asynchronous job queues for long-running tasks.

Examples:

- API Service → AI Service

- API Service → Document Service
- Worker → AI Service
- AI Service → MCP Service
- All internal requests include:
- Correlation ID
- Service identity
- Trace context

9.18 API Documentation

Every endpoint is documented with:

- Description
- Request schema
- Response schema
- Authentication requirements
- Error codes
- Example requests
- Example responses
- Documentation is generated automatically from FastAPI and published as an interactive OpenAPI interface.

9.19 Versioning Strategy

Breaking changes require a new API version.

Non-breaking additions are introduced within the current version.

Deprecated endpoints remain available for a defined transition period before removal.

Chapter Summary

This chapter defined the API architecture of EKOA, including REST conventions, authentication, streaming, service contracts, error handling, pagination, security, and versioning. These contracts provide a stable interface for all clients and internal services, enabling parallel development and long-term maintainability.

Chapter 10: AI Architecture (LLMs, RAG, LangChain, LangGraph & Multi-Agent System)

10.1 Introduction

The AI subsystem is the intelligence layer of EKOA. Rather than acting as a single conversational model, it orchestrates specialized components that retrieve knowledge, reason over enterprise data, invoke external tools, and automate business workflows.

The architecture is built around four principles:

- Retrieval before generation.
- Model independence.
- Agent specialization.
- Explainable AI responses.
- This allows EKOA to provide accurate, contextual, and auditable responses suitable for enterprise use.

10.2 AI Architecture Overview

The AI layer consists of:

- AI Gateway
- Intent Classifier
- Prompt Manager
- Model Router
- LangGraph Runtime
- LangChain Components
- RAG Engine
- Memory Manager
- Tool Executor
- MCP Client
- Agent Registry
- Response Generator
- Guardrails
- LangSmith Observability

- Each component has a single responsibility and communicates through well-defined interfaces.

10.3 AI Request Lifecycle

When a user submits a request, the following sequence occurs:

- Authenticate the request.
- Classify user intent.
- Detect the required capabilities.
- Select the appropriate execution path.
- Retrieve contextual knowledge if needed.
- Choose the optimal language model.
- Execute agents or tools.
- Validate the response.
- Stream the answer with citations and metadata.

This adaptive lifecycle avoids unnecessary processing for simple requests while enabling sophisticated workflows for complex tasks.

10.4 Intent Classification

The Intent Classifier determines how a request should be handled.

Example categories:

- Question answering
- Knowledge search
- Document summarization
- Document comparison
- Research
- Code assistance
- Workflow execution
- Analytics
- Translation
- Email drafting
- Report generation

The classification result determines whether the request can be handled by a simple retrieval flow or requires multi-agent orchestration.

10.5 Model Router

The Model Router selects the most appropriate LLM based on task complexity, cost, privacy, and administrator policies.

Initial providers:

- DeepSeek (general reasoning)

- Ollama (local/private inference)
- Google Gemini (general-purpose free tier)
- Future providers:
- OpenAI
- Anthropic Claude
- Mistral
- Azure OpenAI
- Grok
- Routing factors include:
- Estimated token count.
- Required reasoning depth.
- Availability.
- Latency.
- Cost budget.
- Organizational policy.
- Data sensitivity.
- If a preferred provider is unavailable, the router automatically falls back to the next eligible model.

10.6 Prompt Management

Prompt engineering is centralized through a Prompt Manager.

Each prompt is:

- Versioned.
- Tested.
- Reusable.
- Parameterized.
- Environment-aware.
- Prompt categories include:
- Retrieval prompts
- Summarization prompts
- Research prompts
- Code review prompts
- Workflow planning prompts
- Report generation prompts
- Changes to prompts are tracked with version history and can be rolled back if necessary.

10.7 Retrieval-Augmented Generation (RAG)

The RAG engine grounds AI responses in enterprise knowledge rather than relying solely on model parameters.

Pipeline:

- Receive query.

- Generate query embedding.
- Search Qdrant.
- Apply metadata filters.
- Rerank retrieved chunks.
- Assemble context.
- Send context to the selected LLM.
- Generate response.
- Return citations.
- This approach improves factual accuracy and reduces hallucinations.

10.8 Hybrid Retrieval

Retrieval combines multiple techniques:

- Semantic vector search.
- Keyword search.
- Metadata filtering.
- Optional reranking.
- Examples of metadata filters:
- Organization.
- Workspace.
- Department.
- Document type.
- Language.
- Sensitivity level.
- Date range.
- Hybrid retrieval ensures relevant results even when terminology differs between documents.

10.9 LangChain Components

LangChain provides reusable building blocks, including:

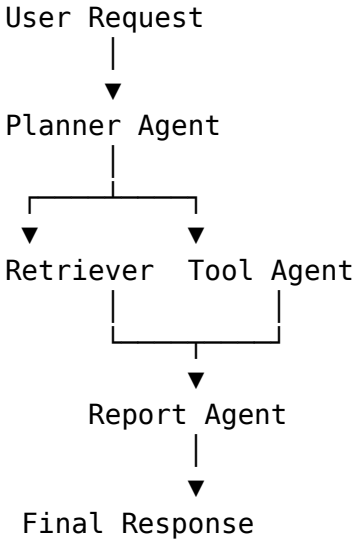
- Prompt templates.
- Document loaders.
- Text splitters.
- Retrievers.
- Embedding interfaces.
- Output parsers.
- Tool abstractions.
- Callback handlers.
- These abstractions reduce boilerplate and standardize interactions with LLMs and external services.

10.10 LangGraph Runtime

LangGraph orchestrates complex, stateful AI workflows.

Core capabilities:

- Directed graph execution.
- Conditional branching.
- Parallel execution.
- Persistent state.
- Retry logic.
- Human approval checkpoints.
- Example flow:



This graph-based execution model is well suited for enterprise automation.

10.11 Multi-Agent Architecture

Rather than relying on a single general-purpose agent, EKOA employs specialized agents coordinated by a central orchestrator.

Proposed agents:

Agent	Responsibility
Coordinator Agent	Overall orchestration and task delegation
Planner Agent	Breaks complex requests into executable steps
Retriever Agent	Retrieves relevant knowledge from RAG
Research Agent	Synthesizes information across sources
Document Agent	Analyzes and compares documents
Workflow Agent	Executes business workflows
Integration Agent	Calls external systems via APIs or MCP
Code Agent	Assists with software engineering tasks

Agent	Responsibility
Analytics Agent	Produces metrics and business insights
Reporting Agent	Formats structured reports
Notification Agent	Sends notifications
QA Agent	Validates AI outputs before delivery

Each agent is independently testable and can evolve without affecting others.

10.12 Memory Management

Memory is separated into distinct layers:

- Short-Term Memory Maintains the current conversation context.
- Long-Term Memory Stores persistent user or organizational preferences where permitted.
- Semantic Memory Uses vector embeddings to retrieve relevant historical knowledge.
- Workflow Memory Tracks state across long-running executions.
- This separation prevents context overload and improves scalability.

10.13 Tool Calling

Agents can invoke tools when additional capabilities are required.

Examples:

- Search enterprise knowledge.
- Query GitHub.
- Create Jira issues.
- Search Notion.
- Read Google Drive files.
- Execute SQL (restricted).
- Generate charts.
- Invoke MCP tools.
- All tool invocations are validated against permissions before execution.

10.14 Guardrails

To reduce unsafe or incorrect behavior, the AI layer enforces guardrails:

- Input validation.
- Prompt injection detection.
- Sensitive data masking.
- Policy enforcement.
- Tool permission checks.
- Output validation.

- Human approval for high-impact actions.
- Guardrails are applied before and after model execution.

10.15 Human-in-the-Loop

Certain actions require explicit approval before execution, such as:

- Sending emails.
- Creating external tickets.
- Deleting documents.
- Updating organizational policies.
- Executing administrative workflows.
- Approval requests are integrated into the workflow engine.

10.16 AI Observability

The AI subsystem is instrumented with LangSmith and OpenTelemetry.

Captured metrics include:

- Prompt version.
- Model used.
- Token usage.
- Latency.
- Retrieval quality.
- Tool invocations.
- Agent execution paths.
- Error rates.
- Estimated inference cost.
- These metrics support optimization and debugging.

10.17 Cost Optimization

The AI layer minimizes operational cost by:

- Selecting smaller models for simple tasks.
- Using local inference via Ollama when appropriate.
- Caching embeddings.
- Reusing retrieved context.
- Limiting context windows.
- Avoiding redundant model calls.
- Streaming responses to improve perceived performance.

10.18 AI Safety

The platform enforces responsible AI practices by:

- Restricting access to sensitive data.
- Logging AI decisions.
- Providing source citations.
- Indicating confidence where appropriate.
- Preventing unauthorized tool execution.
- Allowing administrators to define AI usage policies.

Chapter Summary

This chapter defined the complete AI architecture of EKOA, including adaptive request routing, provider-agnostic LLM integration, retrieval-augmented generation, LangChain components, LangGraph orchestration, multi-agent collaboration, memory management, tool calling, guardrails, and observability. Together, these capabilities form the intelligent core of the platform while maintaining enterprise standards for accuracy, security, and governance.

Chapter 11: Enterprise Knowledge Ingestion & RAG Pipeline

11.1 Introduction

The Knowledge Ingestion Pipeline transforms raw enterprise information into structured, searchable, and AI-ready knowledge.

Instead of treating every file as plain text, EKOA performs intelligent preprocessing to preserve document structure, metadata, and context.

The pipeline is designed for:

- Scalability
- Accuracy
- Incremental updates
- Version awareness
- Enterprise security
- Explainable retrieval

11.2 Supported Knowledge Sources

The system shall support ingestion from multiple content sources.

Documents

- PDF
- DOCX
- TXT
- Markdown
- CSV
- JSON
- XML
- Images PNG
- JPG
- JPEG
- TIFF
- BMP
- OCR converts images into searchable text.

- Audio Supported examples:
- MP3
- WAV
- M4A
- AAC
- Audio is transcribed before indexing.
- Video Supported examples:
- MP4
- AVI
- MOV
- MKV
- The pipeline extracts:
- Speech transcript
- Scene timestamps
- Optional key-frame metadata
- Enterprise Sources GitHub repositories
- Notion
- Confluence
- Google Drive
- OneDrive
- SharePoint
- Slack exports
- Microsoft Teams exports
- REST APIs

11.3 Knowledge Ingestion Architecture

Every knowledge source follows a standardized pipeline:

```
Source
   |
Upload / Connector
   |
Virus Scan
   |
Content Extraction
   |
Metadata Extraction
   |
Language Detection
   |
Document Classification
   |
Chunking
   |
Embedding Generation
   |
```

```
Vector Storage
   |
Metadata Storage
   |
Knowledge Ready
```

Each stage produces structured outputs and logs execution details.

11.4 File Parsing Layer

Different parsers are used depending on file type.

PDF

Extract:

- Text
- Tables
- Images
- Headers
- Footers
- Page numbers
- Recommended libraries:
- PyMuPDF
- pdfplumber
- Unstructured
- DOCX Extract:
- Headings
- Paragraphs
- Tables
- Lists
- Comments (optional)
- Recommended:
- python-docx
- Unstructured
- CSV Extract:
- Columns
- Data types
- Summary statistics
- Headers
- Large CSV files may be indexed by logical sections rather than entire rows.
- Images Pipeline:
- Image
- ↓
- OCR
- ↓
- Layout Detection
- ↓
- Text Extraction

- ↓
- Metadata
- ↓
- Index
- Recommended:
- PaddleOCR
- Tesseract (fallback)
- Audio Pipeline:
- Audio
- ↓
- Voice Activity Detection
- ↓
- Transcription
- ↓
- Speaker Diarization (optional)
- ↓
- Timestamp Alignment
- ↓
- Chunking
- Recommended:
- Whisper
- Video Pipeline:
- Video
- ↓
- Audio Extraction
- ↓
- Transcription
- ↓
- Frame Sampling
- ↓
- Optional Image Captioning
- ↓
- Timeline Metadata
- ↓
- Index

11.5 Metadata Enrichment

Each knowledge item receives enriched metadata.

Standard metadata:

- Organization
- Workspace
- Department
- Owner
- Source
- File type

- Language
- Created date
- Modified date
- Version
- Sensitivity level
- Derived metadata:
- Topics
- Named entities
- Keywords
- Summary
- Reading time
- Document category
- Rich metadata enables more precise retrieval.

11.6 Chunking Strategy

A fixed chunk size is not suitable for all content.

EKOA supports multiple chunking strategies.

Recursive Text Chunking

Best for:

- Articles
- Policies
- Documentation
- Semantic Chunking Splits based on meaning rather than character count.
- Best for:
- Technical manuals
- Research papers
- Contracts
- Structural Chunking Uses document hierarchy:
- Chapters
- Sections
- Headings
- Tables
- This preserves context for long documents.
- Time-Based Chunking Used for:
- Audio
- Video
- Chunks are aligned with timestamps to support citation.

11.7 Embedding Generation

Embeddings convert textual content into vector representations suitable for semantic search.

The embedding layer is abstracted to support multiple providers.

Initial models may include:

- BAAI/bge-large-en-v1.5
- BAAI/bge-m3 (multilingual)
- nomic-embed-text (local with Ollama)
- Selection depends on language requirements, hardware availability, and retrieval performance.

11.8 Incremental Indexing

Re-indexing entire repositories is inefficient.

EKOA tracks:

- New documents
- Updated documents
- Deleted documents
- Moved documents
- Only changed content is reprocessed.
- Benefits:
- Lower compute cost
- Faster synchronization
- Reduced downtime

11.9 Version-Aware Retrieval

When multiple versions of a document exist, retrieval can be configured to:

- Use only the latest version.
- Search all versions.
- Compare versions.
- Filter by date range.
- This is especially useful for policies, contracts, and technical documentation.

11.10 Hybrid Retrieval Pipeline

The retrieval process combines several techniques:

- Query preprocessing.
- Query embedding.
- Vector search in Qdrant.
- Keyword search.
- Metadata filtering.
- Result merging.
- Reranking.
- Context assembly.
- LLM response generation.
- This approach improves both recall and precision.

11.11 Reranking

Initial retrieval often returns relevant but imperfect results.

A reranker scores retrieved chunks using a cross-encoder model before passing them to the LLM.

Benefits:

- Better relevance
- Reduced hallucinations
- Higher answer quality

11.12 Context Assembly

The Context Builder selects and organizes retrieved chunks.

Responsibilities:

- Remove duplicates.
- Respect token limits.
- Preserve document order where beneficial.
- Group related sections.
- Include metadata for citations.
- This ensures the model receives coherent and relevant context.

11.13 Citation Engine

Every AI response should include traceable references.

Citations contain:

- Document title
- Version
- Section or page
- Timestamp (for audio/video)
- Source system
- This improves transparency and user trust.

11.14 Retrieval Evaluation

Retrieval quality should be measured continuously.

Suggested metrics:

Metric	Purpose
Precision@k	Measures relevance of top results
Recall@k	Measures coverage of relevant documents

Metric	Purpose
Mean Reciprocal Rank (MRR)	Evaluates ranking quality
nDCG	Measures usefulness of ranking order
Retrieval Latency	Measures search performance

Evaluation datasets should include representative enterprise queries.

11.15 Knowledge Refresh

The platform supports scheduled synchronization.

Examples:

- Every 15 minutes
- Hourly
- Daily
- Weekly
- Manual trigger
- Connectors notify the ingestion pipeline of changes where supported.

11.16 Security During Ingestion

Security controls include:

- Virus scanning.
- File type validation.
- Size limits.
- Access permission checks.
- Metadata sanitization.
- Encryption of stored files.
- Audit logging of ingestion events.
- Sensitive documents retain their original access permissions throughout indexing.

11.17 Failure Handling

If a pipeline stage fails:

- The failure is logged.
- The job is marked as retryable or terminal.
- Retry policies are applied.
- Administrators receive alerts for repeated failures.
- Successfully completed stages are not repeated unnecessarily.

11.18 Future Enhancements

The ingestion pipeline is designed to support future capabilities such as:

- Knowledge graph construction.
- Automatic ontology generation.
- Entity linking across documents.
- Multimodal embeddings.
- Image and diagram understanding.
- Structured table retrieval.
- Domain-specific parsers.

Chapter Summary

This chapter defined the complete knowledge ingestion and retrieval architecture for EKOA, including document parsing, metadata enrichment, adaptive chunking, embedding generation, hybrid retrieval, reranking, citation management, incremental indexing, and retrieval evaluation. These capabilities ensure that enterprise knowledge is transformed into accurate, searchable context for AI reasoning while maintaining traceability and governance.

Chapter 12: Multi-Agent System & LangGraph Workflow Engine

12.1 Introduction

The Multi-Agent System (MAS) is the orchestration layer responsible for executing complex enterprise tasks.

Instead of assigning every request to a single LLM, EKOA decomposes work into specialized responsibilities handled by independent AI agents coordinated through LangGraph.

This architecture improves:

- Scalability
- Explainability
- Reusability
- Accuracy
- Cost efficiency
- Fault isolation
- Each agent has a clearly defined purpose, communicates through structured state, and can be independently tested and optimized.

12.2 Why Multi-Agent?

Enterprise requests are often composed of multiple subtasks.

Example request:

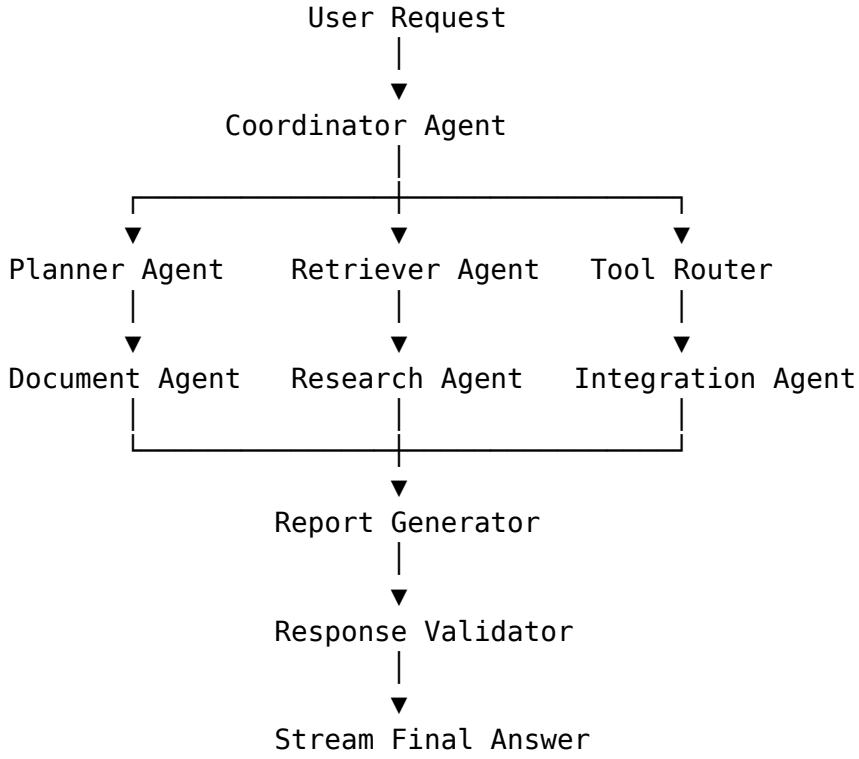
“Compare the latest HR policy with last year’s version, summarize the differences, create a presentation for managers, draft an announcement email, and open Jira tasks for required policy updates.”

A single LLM would struggle to manage planning, retrieval, reasoning, formatting, and external integrations consistently.

Instead, EKOA coordinates multiple specialized agents to complete the task collaboratively.

12.3 Multi-Agent Architecture

The orchestration model is built around a central Coordinator Agent.



The Coordinator manages the overall execution while individual agents focus on their domain expertise.

12.4 Agent Registry

All available agents are registered in a central registry.

Each registration includes:

- Agent name
- Version
- Description
- Supported tasks
- Required tools
- Input schema
- Output schema
- Retry policy
- Timeout
- Cost profile
- Preferred LLM
- The registry allows new agents to be added without modifying the orchestration engine.

12.5 Coordinator Agent

The Coordinator Agent is the entry point for all AI requests.

Responsibilities:

- Receive user request
- Validate context
- Request planning
- Launch workflow
- Monitor execution
- Aggregate results
- Handle failures
- Stream progress
- The Coordinator never performs domain-specific reasoning itself; it delegates work to specialized agents.

12.6 Planner Agent

The Planner Agent decomposes complex requests into executable tasks.

Example:

- User request:
- “Analyze sales performance, identify trends, summarize customer feedback, and prepare a report.”
- Execution plan:
- Retrieve sales data.
- Retrieve customer feedback.
- Analyze trends.
- Generate report.
- Format output.
- The plan is represented as a directed graph executed by LangGraph.

12.7 Retriever Agent

Responsibilities:

- Search Qdrant.
- Apply metadata filters.
- Execute keyword search.
- Merge retrieval results.
- Invoke reranker.
- Return ranked context.
- The Retriever does not generate natural language; it only supplies relevant knowledge.

12.8 Document Agent

Specializes in document understanding.

Capabilities:

- Summarization
- Comparison
- Policy analysis
- Contract review
- Table extraction
- Timeline generation
- Version comparison
- This specialization allows document-related prompts and models to be optimized independently.

12.9 Research Agent

Combines information from multiple sources.

Responsibilities:

- Cross-document synthesis
- Conflict detection
- Gap identification
- Trend analysis
- Citation management
- Outputs are structured and evidence-backed.

12.10 Integration Agent

Handles interactions with external systems.

Supported targets include:

- GitHub
- Jira
- Slack
- Microsoft Teams
- Notion
- Google Drive
- Outlook
- MCP tools
- Custom enterprise APIs
- The Integration Agent never bypasses authorization or approval policies.

12.11 Code Agent

Supports engineering teams by:

- Reviewing code
- Explaining implementations
- Suggesting refactoring
- Generating documentation
- Producing test cases
- Summarizing pull requests
- When necessary, it collaborates with the Integration Agent to retrieve repository data.

12.12 Analytics Agent

Processes structured business data.

Capabilities:

- KPI calculation
- Trend analysis
- Forecast preparation
- Dashboard summarization
- Metric explanation
- Outputs can be formatted as tables, charts (through approved tools), or narrative reports.

12.13 Reporting Agent

Transforms intermediate results into polished outputs.

Supported formats:

- Executive summaries
- Technical reports
- Emails
- Meeting notes
- Action plans
- Markdown
- HTML
- PDF-ready structures
- Formatting is separated from reasoning to improve consistency.

12.14 Notification Agent

Sends workflow updates through configured channels.

Supported channels:

- In-app notifications
- Email
- Slack
- Microsoft Teams
- Notifications respect user preferences and organizational policies.

12.15 Quality Assurance (QA) Agent

Before results are returned, the QA Agent performs validation checks.

Examples:

- Verify citations exist.
- Detect unsupported claims.
- Check required sections.
- Confirm workflow completion.
- Validate output schema.
- If validation fails, the Coordinator may request corrections from upstream agents.

12.16 Shared Workflow State

LangGraph maintains a shared state object throughout execution.

Example state elements:

- Request ID
- User ID
- Organization ID
- Conversation ID
- Current task list
- Retrieved documents
- Intermediate summaries
- Tool outputs
- Approval status
- Execution logs
- This shared state enables collaboration without tightly coupling agents.

12.17 Graph Execution Model

LangGraph supports several execution patterns:

- Sequential Tasks execute one after another.
- Parallel Independent tasks run simultaneously.
- Example:
- Retrieve documents.
- Fetch GitHub data.
- Query Jira.

- These can execute concurrently to reduce latency.
- Conditional Execution path depends on intermediate results.
- Example:
- If no relevant documents are found:
- Skip document analysis.
- Ask the user for clarification.
- Looping Certain nodes may retry until predefined conditions are satisfied or retry limits are reached.

12.18 Human Approval Nodes

Certain workflows pause until human approval is received.

Examples:

- Send an external email.
- Create Jira issues.
- Delete documents.
- Modify organizational settings.
- Trigger high-impact automations.
- Approval status becomes part of the workflow state.

12.19 Failure Recovery

The orchestration engine supports robust recovery.

Failure handling strategies include:

- Automatic retries for transient errors.
- Alternative execution paths.
- Model fallback.
- Tool fallback.
- Partial completion with user notification.
- Workflow checkpoint restoration.
- These mechanisms increase resilience without requiring users to restart long-running tasks.

12.20 Long-Running Workflows

Some enterprise processes may take minutes or hours.

Examples:

- Processing thousands of documents.
- Large repository analysis.
- Organization-wide reporting.
- LangGraph persists workflow state so execution can resume after interruptions.
- Users can monitor progress and receive notifications upon completion.

12.21 Agent Communication Protocol

Agents communicate only through structured messages and shared workflow state.

Messages include:

- Sender
- Receiver
- Correlation ID
- Payload
- Status
- Timestamp
- This approach simplifies debugging and auditing.

12.22 Workflow Templates

Administrators can define reusable workflow templates.

Examples:

- New employee onboarding.
- Weekly engineering report.
- Policy review.
- Security audit.
- Release preparation.
- Incident postmortem.
- Templates can be customized per organization.

12.23 Agent Performance Metrics

Each agent records:

- Execution time
- Model used
- Token usage
- Tool invocations
- Success rate
- Retry count
- Cost estimate
- Validation outcomes
- These metrics support optimization and capacity planning.

12.24 Security & Governance

Every agent operates under the requesting user’s permissions.

Key principles:

- No privilege escalation.

- Least privilege access.
- Audit every tool invocation.
- Validate external actions.
- Enforce organization policies.
- Mask sensitive information where required.
- The Coordinator ensures all downstream actions comply with these constraints.

Chapter Summary

This chapter defined the orchestration engine at the heart of EKOA. Using LangGraph, specialized AI agents collaborate through shared workflow state, structured communication, and adaptive execution patterns. The result is a flexible, resilient, and governable multi-agent system capable of supporting sophisticated enterprise workflows while maintaining security, observability, and explainability.

Chapter 13: Model Context Protocol (MCP) Architecture

13.1 Introduction

The Model Context Protocol (MCP) provides a standardized mechanism for AI models to discover, access, and invoke external capabilities.

EKOA implements MCP as a core architectural layer rather than as an optional integration.

The platform supports both:

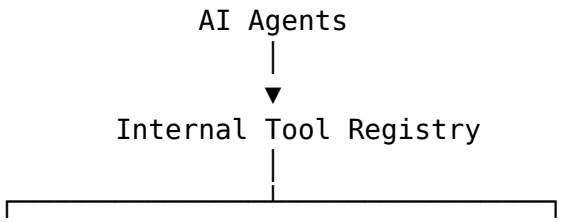
- MCP Client — to consume external tools and resources.
- MCP Server — to expose EKOA's own capabilities.
- This enables interoperability with AI assistants, IDEs, enterprise automation systems, and future MCP-compatible platforms.

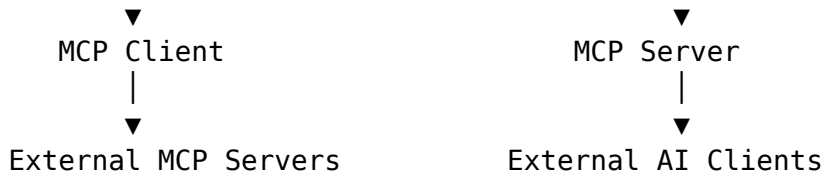
13.2 MCP Objectives

The MCP implementation is designed to achieve the following objectives:

- Standardize tool access.
- Reduce custom integration code.
- Enable dynamic capability discovery.
- Simplify AI tool orchestration.
- Support provider-independent AI workflows.
- Enforce centralized authorization.
- Improve interoperability.

13.3 High-Level Architecture





The Tool Registry sits at the center of all tool interactions.

13.4 MCP Client

The MCP Client enables EKOA to connect to external MCP servers.

Responsibilities:

- Server registration
- Authentication
- Capability discovery
- Tool invocation
- Resource retrieval
- Prompt retrieval
- Session management
- Retry handling
- Supported transports:
- STDIO
- HTTP
- The architecture should allow future WebSocket transport support.

13.5 External MCP Servers

Examples include:

- GitHub MCP Server
- Filesystem MCP Server
- PostgreSQL MCP Server
- Browser Automation MCP Server
- Slack MCP Server
- Jira MCP Server
- Custom enterprise MCP servers
- Administrators can enable or disable servers per organization.

13.6 MCP Server

The EKOA MCP Server exposes internal platform capabilities to external AI systems.

Examples of exposed tools:

- Search knowledge base
- Retrieve documents

- Execute workflows
- Query analytics
- List organizations (authorized users)
- Retrieve prompt templates
- Access workflow status
- Only explicitly published capabilities are available externally.

13.7 Tool Registry

Every tool is registered through a centralized Tool Registry.

Each tool definition contains:

- Unique identifier
- Human-readable name
- Description
- Version
- Input schema
- Output schema
- Required permissions
- Timeout
- Retry policy
- Cost category
- Tags
- The Tool Registry serves as the authoritative catalog for both internal and external tool access.

13.8 Tool Categories

Tools are grouped into logical categories.

Knowledge Tools

- Search documents
- Retrieve document
- Compare documents
- List collections
- Integration Tools GitHub
- Jira
- Slack
- Notion
- Google Drive
- Outlook
- Workflow Tools Execute workflow
- Pause workflow
- Resume workflow
- Cancel workflow
- Analytics Tools KPI reports

- Usage statistics
- AI metrics
- Audit summaries
- Administration Tools Restricted to privileged users:
- User management
- Role management
- Organization settings
- AI provider configuration

13.9 Resource Exposure

MCP resources provide structured information that AI clients can consume.

Examples:

- Organization metadata
- Workspace list
- Document catalog
- Workflow definitions
- Prompt library
- Analytics snapshots
- Resources are read-only unless explicitly exposed as writable through approved tools.

13.10 Prompt Exposure

Prompt templates may be exposed through MCP for authorized clients.

Examples:

- Document summarization
- Policy comparison
- Incident analysis
- Code review
- Executive reporting
- Each prompt includes:
- Version
- Description
- Parameters
- Expected outputs
- This enables consistent prompting across internal and external clients.

13.11 Tool Execution Lifecycle

When a tool is invoked:

- Validate authentication.
- Verify authorization.

- Validate input schema.
- Execute the tool.
- Capture execution metrics.
- Log audit information.
- Return structured results.
- Errors are standardized and include correlation IDs.

13.12 Capability Discovery

When connecting to an MCP server, EKOA retrieves the server's advertised capabilities.

Examples:

- Available tools
- Resources
- Prompt templates
- Supported transports
- Authentication methods
- Capabilities are cached and refreshed periodically.

13.13 Authentication & Authorization

Every MCP interaction is authenticated.

Supported mechanisms:

- OAuth
- API keys
- JWT (internal)
- Service credentials
- Authorization checks ensure:
- Organization isolation
- Role-based permissions
- Tool-specific access rules
- Resource ownership validation

13.14 Transport Layer

Supported transports:

- STDIO Used for local integrations and developer tools.
- HTTP Used for remote enterprise deployments.
- Future support:
- WebSocket transport
- gRPC (internal experimentation)
- The transport layer is abstracted so new protocols can be added without changing tool implementations.

13.15 Error Handling

Standard error categories include:

- Authentication failed
- Authorization denied
- Validation error
- Tool unavailable
- Timeout
- Rate limit exceeded
- Internal execution error
- Clients receive structured error responses with actionable messages where appropriate.

13.16 Security Considerations

The MCP layer enforces:

- Least privilege.
- Input validation.
- Output sanitization.
- Audit logging.
- Rate limiting.
- Tool isolation.
- Secure secret storage.
- External tools never gain direct database access; all interactions pass through the Tool Registry and permission checks.

13.17 Observability

Every MCP operation records:

- Request ID
- Tool name
- Caller identity
- Latency
- Success/failure
- Retry count
- Resource usage
- These metrics integrate with the platform’s observability stack for monitoring and debugging.

13.18 Extensibility

Organizations can develop custom MCP-compatible tools by implementing the standard tool interface.

New tools automatically benefit from:

- Discovery
- Authentication
- Authorization
- Auditing
- Monitoring
- No changes to the orchestration engine are required.

13.19 Example Workflow

Example request:

- “Create a Jira issue based on the findings in the latest security audit.”
- Execution flow:
- Planner Agent decomposes the request.
- Retriever Agent retrieves the audit report.
- Reporting Agent summarizes findings.
- Integration Agent invokes the Jira tool through the Tool Registry.
- MCP Client communicates with the Jira MCP Server.
- Jira issue is created (after any required human approval).
- Result is returned with the created issue reference.

13.20 Future Roadmap

Potential future enhancements include:

- Dynamic tool marketplaces.
- Organization-specific MCP plugins.
- Tool capability scoring.
- Semantic tool search.
- AI-generated tool compositions.
- Cross-organization federation (with explicit trust relationships).

Chapter Summary

This chapter defined the Model Context Protocol architecture within EKOA. By implementing both client and server roles around a centralized Tool Registry, the platform achieves standardized, secure, and extensible interoperability with external AI ecosystems while maintaining governance, observability, and permission enforcement.

Chapter 14: Security Architecture & Identity Management

14.1 Introduction

Security is a foundational concern throughout EKOA rather than an isolated subsystem. The platform adopts a Zero Trust approach in which every request, user, service, and tool invocation is authenticated, authorized, validated, and audited.

The security architecture protects:

- Enterprise knowledge
- User identities
- AI interactions
- External integrations
- Administrative operations
- Stored files
- API communications

14.2 Security Principles

The platform follows these principles:

- Zero Trust
- Least privilege
- Defense in depth
- Secure by default
- Explicit authorization
- Complete auditability
- Strong identity verification
- Encryption everywhere
- Secure software supply chain

14.3 Identity Architecture

Identity is managed centrally.

Authentication providers include:

- Email & Password
- Google OAuth
- Microsoft OAuth
- GitHub OAuth
- Future support may include:
- SAML 2.0
- OpenID Connect enterprise providers
- Azure Active Directory
- Okta
- Auth0
- All providers are normalized into a unified internal user model.

14.4 Authentication Flow

Standard authentication process:

```
User
 │
 ▼
Identity Provider
 │
 ▼
JWT Access Token
 │
 ▼
Refresh Token
 │
 ▼
Authenticated Session
```

Access tokens are short-lived, while refresh tokens are securely stored and rotated.

14.5 JWT Strategy

Two tokens are issued:

- Access Token Contains:
- User ID
- Organization ID
- Role identifiers
- Expiration
- Token ID
- Lifetime:
- Approximately 15–30 minutes
- Refresh Token Stored securely and used to obtain new access tokens without requiring the user to log in again.
- Refresh tokens are rotated after each successful use.

14.6 Session Management

Each active session records:

- Device identifier
- Browser information
- IP address
- Login time
- Last activity
- Token ID
- Session status
- Users can view and revoke active sessions from their account settings.

14.7 Authorization Model

Authorization combines RBAC and ABAC.

Role-Based Access Control (RBAC) Example roles:

- Super Admin
- Organization Admin
- Workspace Admin
- Manager
- Employee
- Guest
- Each role grants a baseline permission set.
- Attribute-Based Access Control (ABAC) Additional policies evaluate:
- Organization
- Department
- Team
- Workspace
- Document sensitivity
- Resource ownership
- Time restrictions
- Device trust level
- A request must satisfy both role permissions and attribute policies.

14.8 Permission Model

Permissions are expressed as actions on resources.

Examples:

- documents.read
- documents.write
- documents.delete
- workflows.execute
- users.manage

- integrations.configure
- ai.chat
- analytics.view
- Permissions are assigned through roles and evaluated alongside ABAC rules.

14.9 Multi-Tenant Isolation

Every request is scoped to a single organization.

Isolation mechanisms include:

- Organization identifiers in all business entities.
- Authorization checks on every query.
- Filtered vector search by organization.
- Segregated object storage paths.
- Organization-aware caches.
- The design prevents data leakage across tenants.

14.10 Secure File Handling

Uploaded files pass through a secure ingestion process:

- File type validation.
- Size validation.
- Virus scanning.
- Safe storage.
- Metadata extraction.
- Indexing.
- Executable files and unsupported formats are rejected.

14.11 Secret Management

Sensitive secrets include:

- OAuth credentials
- API keys
- Database passwords
- Encryption keys
- Service tokens
- Secrets are never stored in source code or version control.

In development, they are provided through environment variables. In production, they should be managed using a dedicated secret management solution.

14.12 Encryption

Data in Transit

- All communications use TLS.
- Examples:
- Browser ↔ API
- API ↔ AI Service
- Service ↔ Database (where supported)
- External integrations
- Data at Rest Sensitive data is encrypted before storage where appropriate.
- This includes:
- Refresh tokens
- API credentials
- Integration secrets

14.13 Password Security

Passwords are:

- Never stored in plaintext.
- Hashed using Argon2id (preferred) or bcrypt.
- Validated against configurable password policies.
- Optional password requirements include:
- Minimum length
- Complexity
- Password history
- Breach detection

14.14 API Security

The API layer enforces:

- JWT validation.
- Input validation.
- Request size limits.
- Rate limiting.
- CORS policies.
- Security headers.
- CSRF protection where applicable.
- Idempotency support.
- All requests receive a correlation ID for traceability.

14.15 AI Security

AI introduces unique risks that require additional controls.

The platform implements:

- Prompt injection detection.
- Prompt template isolation.

- Output validation.
- Sensitive data masking.
- Tool permission enforcement.
- Context filtering.
- High-risk actions require human approval before execution.

14.16 Integration Security

Each external integration is isolated.

Controls include:

- OAuth token encryption.
- Permission scoping.
- Secure token refresh.
- Revocation support.
- Organization-level approval.
- Unused integrations can be disabled without affecting other services.

14.17 Audit Logging

Security-relevant events are logged.

Examples:

- Login
- Logout
- Failed login
- Password reset
- Role change
- File deletion
- Workflow approval
- AI tool invocation
- Integration configuration
- Audit entries include:
- Actor
- Action
- Resource
- Timestamp
- IP address
- Correlation ID
- Audit logs are immutable.

14.18 Threat Model

Key threats include:

- Credential theft

- Prompt injection
- Cross-site scripting (XSS)
- Cross-site request forgery (CSRF)
- SQL injection
- SSRF
- Malware uploads
- Privilege escalation
- Tenant isolation failures
- Denial-of-service attacks
- Each threat is addressed through layered controls, validation, and monitoring.

14.19 Incident Response

Security incidents follow a structured process:

- Detection.
- Containment.
- Investigation.
- Eradication.
- Recovery.
- Post-incident review.
- Alerts integrate with the monitoring stack to reduce response time.

14.20 Compliance Considerations

The architecture is designed to align with common enterprise expectations.

Relevant frameworks include:

- GDPR principles (where applicable)
- SOC 2 security controls
- ISO/IEC 27001 information security practices
- Compliance implementation depends on deployment and organizational requirements.

14.21 Security Testing

Security validation includes:

- Static Application Security Testing (SAST)
- Dependency vulnerability scanning
- Dynamic Application Security Testing (DAST)
- Penetration testing
- API security testing
- Authentication testing
- Authorization testing
- Security testing is integrated into the CI/CD pipeline.

14.22 Monitoring & Alerting

Security metrics include:

- Failed login attempts
- Token validation failures
- Suspicious API usage
- Excessive rate limit violations
- Privilege changes
- AI policy violations
- Integration failures
- Critical events generate immediate alerts for administrators.

Chapter Summary

This chapter defined the security architecture of EKOA, including identity management, hybrid RBAC/ABAC authorization, multi-tenant isolation, secure file handling, secret management, encryption, AI-specific safeguards, audit logging, threat modeling, and compliance considerations. These controls establish a strong security foundation suitable for enterprise AI applications while preserving usability and extensibility.

Chapter 15: DevOps, Infrastructure & Deployment Architecture

15.1 Introduction

The DevOps architecture provides the operational foundation of EKOA, enabling consistent development, testing, deployment, monitoring, and maintenance across environments.

The infrastructure is designed to be:

- Reproducible
- Portable
- Secure
- Observable
- Scalable
- Fault tolerant
- Containerization ensures that all services behave consistently from local development to production deployments.

15.2 Infrastructure Overview

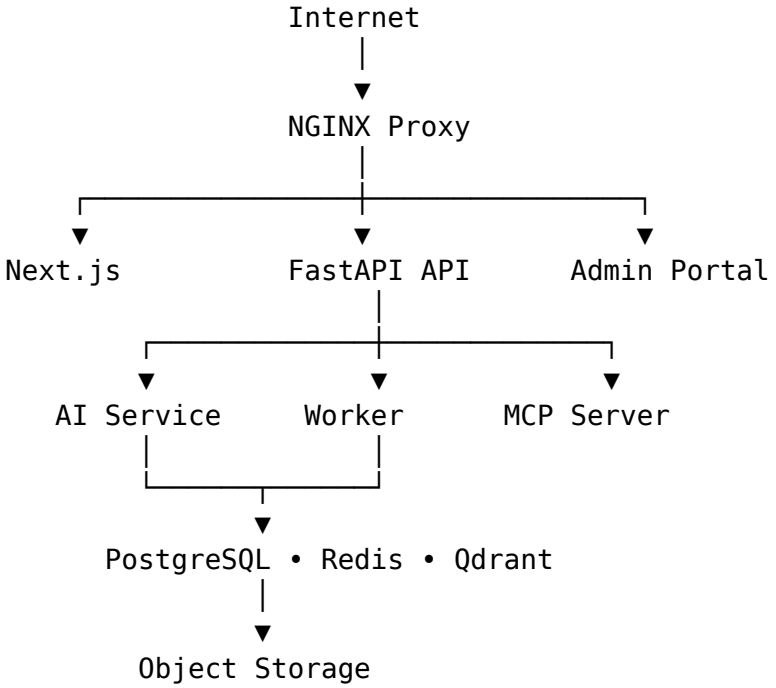
The platform consists of the following runtime services:

Service	Purpose
Next.js Web	User interface
Admin Portal	Administrative interface
FastAPI API	Business logic
AI Service	LangChain, LangGraph, RAG
Worker Service	Background processing
MCP Server	MCP tools and resources
PostgreSQL	Relational database
Qdrant	Vector database
Redis	Cache and queues
Object Storage	Enterprise documents
NGINX	Reverse proxy
Prometheus	Metrics

Service	Purpose
Grafana	Dashboards
Loki	Log aggregation
Tempo	Distributed tracing
LangSmith	AI observability
Sentry	Error tracking

Each component is independently containerized.

15.3 Container Architecture



The reverse proxy terminates HTTPS and routes requests to internal services.

15.4 Development Environment

Development uses Docker Compose.

Core objectives:

- One-command startup.
- Isolated dependencies.
- Consistent environments.
- Fast onboarding.
- Typical command:
- docker compose up –build All required services start automatically.

15.5 Environment Configuration

Separate configuration files are maintained for each environment.

Examples:

- .env.development .env.testing .env.staging .env.production Each service reads only the variables it requires.
- No secrets are committed to version control.

15.6 Networking

Docker networks isolate internal communication.

Example:

- frontend-network
- backend-network
- monitoring-network Only the reverse proxy exposes public ports.
- Databases remain inaccessible from the public internet.

15.7 Persistent Storage

Persistent volumes are used for:

- PostgreSQL
- Qdrant
- Redis (if persistence is enabled)
- Uploaded files
- Logs
- Monitoring data
- Volumes ensure data survives container restarts.

15.8 CI/CD Pipeline

GitHub Actions automates software delivery.

Typical workflow:

- Push
- ↓
- Lint
- ↓
- Unit Tests
- ↓
- Integration Tests
- ↓
- Security Scan

- ↓
- Docker Build
- ↓
- Push Images
- ↓
- Deploy Every stage must succeed before deployment proceeds.

15.9 Continuous Integration

CI validates:

- Code formatting
- Static analysis
- Unit tests
- Integration tests
- Dependency vulnerabilities
- Docker image builds
- This reduces defects before deployment.

15.10 Continuous Deployment

Deployment environments include:

- Development
- Staging
- Production
- Promotion to production requires successful validation and, where appropriate, manual approval.

15.11 Reverse Proxy

NGINX responsibilities include:

- HTTPS termination
- Routing
- Compression
- Caching (selected content)
- Security headers
- Rate limiting
- Static asset delivery
- Future deployments may replace NGINX with Traefik or Envoy if advanced service mesh capabilities are required.

15.12 Monitoring Stack

Operational monitoring combines several tools.

Prometheus

- Collects system and application metrics.
- Grafana Visualizes dashboards.
- Loki Aggregates structured logs.
- Tempo Provides distributed tracing.
- LangSmith Monitors AI execution.
- Sentry Captures application exceptions.
- Together, these tools provide end-to-end visibility into platform behavior.

15.13 Logging Strategy

Applications emit structured JSON logs.

Each log entry includes:

- Timestamp
- Service name
- Log level
- Correlation ID
- User ID (when applicable)
- Organization ID
- Message
- Sensitive information is never written to logs.

15.14 Distributed Tracing

Every request receives a unique correlation ID.

The ID propagates across services:

- Browser
- ↓
- API
- ↓
- AI Service
- ↓
- Worker
- ↓
- Database Tracing simplifies debugging of complex workflows.

15.15 Health Checks

Each service exposes:

- Liveness endpoint
- Readiness endpoint
- Example endpoints:

- /health/live
- /health/ready Docker and orchestration platforms use these endpoints to detect failures.

15.16 Backup Strategy

Backup policies include:

- PostgreSQL Nightly full backup.
- Continuous WAL archiving.
- Qdrant Regular snapshots.
- Object Storage Versioning.
- Replication.
- Configuration Infrastructure definitions stored in Git.
- Environment secrets stored separately.
- Recovery procedures are tested periodically.

15.17 Disaster Recovery

Recovery priorities:

- Restore databases.
- Restore object storage.
- Start infrastructure.
- Restore AI indexes.
- Validate application integrity.
- Recovery documentation is maintained alongside deployment documentation.

15.18 Scaling Strategy

Horizontal scaling targets:

- API Service
- AI Service
- Worker Service
- MCP Server
- Vertical scaling targets:
- PostgreSQL
- Qdrant
- The architecture supports scaling bottleneck services independently.

15.19 High Availability

Future production deployments may include:

- Multiple API replicas.

- Multiple AI service replicas.
- Load balancing.
- Database replication.
- Redundant object storage.
- Health-based failover.
- These enhancements improve resilience during failures.

15.20 Infrastructure as Code

Infrastructure definitions should be version-controlled.

Examples include:

- Docker Compose files.
- NGINX configuration.
- GitHub Actions workflows.
- Monitoring dashboards.
- Environment templates.
- Future cloud deployments can introduce tools such as Terraform without changing application architecture.

15.21 Release Strategy

Recommended release flow:

- Feature branches.
- Pull requests.
- Automated validation.
- Merge to main.
- Version tagging.
- Container image creation.
- Deployment.
- Post-deployment verification.
- Semantic Versioning (SemVer) is recommended for releases.

15.22 Performance Optimization

Operational optimizations include:

- HTTP compression.
- Static asset caching.
- Database indexing.
- Connection pooling.
- Redis caching.
- Streaming AI responses.
- Lazy loading where appropriate.
- Performance metrics are reviewed regularly to identify bottlenecks.

15.23 Production Readiness Checklist

Before production deployment:

- All tests pass.
- Security scan completed.
- Secrets configured.
- Monitoring enabled.
- Backups verified.
- Health checks operational.
- Documentation updated.
- Rollback plan prepared.

Chapter Summary

This chapter defined the operational architecture of EKOA, covering containerization, networking, environment management, CI/CD, monitoring, logging, tracing, backup strategies, disaster recovery, scaling, and production deployment. The resulting infrastructure supports reliable development and provides a clear migration path from local Docker Compose deployments to enterprise-scale environments.

Chapter 16: Frontend Architecture & User Experience

16.1 Introduction

The frontend is the primary interface through which users interact with EKOA. It is responsible for delivering a responsive, accessible, secure, and intuitive experience while integrating seamlessly with the platform's AI capabilities.

The frontend architecture emphasizes:

- Performance
- Scalability
- Accessibility
- Maintainability
- Real-time interaction
- Enterprise usability

16.2 Technology Stack

Layer	Technology
Framework	Next.js 15 (App Router)
Language	TypeScript
Styling	Tailwind CSS
Components	shadcn/ui
Icons	Lucide React
Forms	React Hook Form + Zod
State Management	Zustand + TanStack Query
Tables	TanStack Table
Charts	Recharts
Animations	Framer Motion
Authentication	JWT + OAuth
Theme	next-themes

16.3 Frontend Directory Structure

```
apps/web/
├── app/
├── features/
├── components/
├── hooks/
├── lib/
├── services/
├── providers/
├── styles/
├── types/
├── middleware.ts
└── public/
```

The features directory contains business-specific functionality, while shared UI elements remain in components.

16.4 Feature Modules

Each feature is self-contained.

Example:

```
features/
├── auth/
│   ├── components/
│   ├── hooks/
│   ├── services/
│   ├── schemas/
│   └── types/
├── chat/
├── knowledge/
├── workflows/
├── analytics/
├── settings/
```

This organization improves modularity and reduces coupling.

16.5 Routing

The application uses the Next.js App Router.

Example routes:

/ /login /dashboard /chat /knowledge /workspaces /workflows /agents /integrations /analytics /settings /admin Route groups and layouts provide consistent navigation and shared UI elements.

16.6 Authentication Flow

The frontend manages authentication through:

- User login.
- Token storage (secure HTTP-only cookies where applicable).
- Protected route middleware.
- Automatic token refresh.
- Session expiration handling.
- Unauthorized users are redirected to the login page.

16.7 Layout System

The application uses nested layouts.

Example:

```
Root Layout
    │
    ├── Public Layout
    │
    ├── Dashboard Layout
    │       ├── Sidebar
    │       ├── Header
    │       └── Content
    │
    └── Admin Layout
```

This reduces duplication and provides a consistent user experience.

16.8 Navigation

Primary navigation includes:

- Dashboard
- AI Chat
- Knowledge Base
- Documents
- Workflows
- Agents
- Integrations
- Analytics
- Settings
- Administration (authorized users)
- Breadcrumbs provide contextual navigation within deeper pages.

16.9 Dashboard

The dashboard acts as the user’s home screen.

Widgets may include:

- Recent conversations
- Recent documents
- Active workflows
- AI usage
- Notifications
- Organization announcements
- System health
- Quick actions
- Widgets are configurable per user.

16.10 AI Chat Interface

The AI chat is the platform’s central workspace.

Features:

- Streaming responses.
- Source citations.
- Markdown rendering.
- Code syntax highlighting.
- Tool execution status.
- Agent activity timeline.
- File attachments.
- Conversation history.
- Suggested follow-up prompts.
- The interface clearly distinguishes between AI-generated content and retrieved sources.

16.11 Knowledge Management

The knowledge section supports:

- Document upload.
- Folder organization.
- Metadata editing.
- Version history.
- Search.
- Filters.
- Bulk actions.
- Preview.
- Processing status is displayed for newly uploaded content.

16.12 Workflow Builder

The workflow interface allows users to:

- Browse templates.
- Create workflows.
- Configure triggers.
- View execution history.
- Monitor running workflows.
- Approve pending actions.
- Future versions may include a visual drag-and-drop editor.

16.13 Real-Time Updates

Real-time communication is provided through WebSockets.

Examples:

- AI streaming tokens.
- Workflow progress.
- Notification updates.
- Background job status.
- Agent execution events.
- The UI reflects progress without requiring manual refresh.

16.14 State Management

State is divided into:

- Server State Managed with TanStack Query.
- Examples:
- Documents
- Users
- Workspaces
- Analytics
- Client State Managed with Zustand.
- Examples:
- Theme
- Sidebar state
- Active conversation
- Draft messages
- This separation avoids unnecessary complexity.

16.15 Forms

All forms use:

- React Hook Form

- Zod validation
- Validation occurs both client-side and server-side.
- Examples:
- Login
- Profile
- Document upload
- Workflow creation
- Integration configuration

16.16 Design System

The UI follows a consistent design language.

Components include:

- Buttons
- Cards
- Tables
- Dialogs
- Forms
- Badges
- Alerts
- Tabs
- Tooltips
- Dropdowns
- Toasts
- Components are built with shadcn/ui and customized to match the platform’s branding.

16.17 Accessibility

The frontend targets WCAG 2.1 AA principles.

Key practices:

- Keyboard navigation.
- Screen reader support.
- Sufficient color contrast.
- Focus indicators.
- Semantic HTML.
- Accessible form labels.
- Accessibility is considered throughout component development.

16.18 Internationalization

The application is designed for multilingual support.

Initial languages:

- English
- Planned future support:
- Arabic
- French
- German
- Spanish
- Localization covers UI text, dates, numbers, and formatting.

16.19 Responsive Design

The interface supports:

- Desktop
- Laptop
- Tablet
- Mobile

Administrative dashboards are optimized primarily for larger screens, while core user workflows remain fully functional on mobile devices.

16.20 Performance Optimization

Frontend performance techniques include:

- Server Components where appropriate.
- Dynamic imports.
- Lazy loading.
- Image optimization.
- Route prefetching.
- Memoization.
- Virtualized tables for large datasets.
- These optimizations reduce load times and improve responsiveness.

16.21 Error Handling

The frontend provides consistent error handling through:

- Error boundaries.
- Global toast notifications.
- Inline validation messages.
- Retry options for recoverable failures.
- Friendly fallback screens.
- Technical details are logged while user-facing messages remain clear and actionable.

16.22 Theme Support

The interface supports:

- Light mode
- Dark mode
- System preference
- Theme selection persists across sessions.

16.23 User Experience Principles

The UX follows these principles:

- Minimize cognitive load.
- Surface relevant information.
- Provide immediate feedback.
- Keep workflows consistent.
- Make AI actions transparent.
- Support progressive disclosure for advanced features.
- These principles help both technical and non-technical users adopt the platform.

16.24 Frontend Testing

Frontend testing includes:

- Component tests.
- Integration tests.
- End-to-end tests.
- Accessibility checks.
- Visual regression testing (future enhancement).
- Testing ensures UI reliability as the platform evolves.

Chapter Summary

This chapter defined the frontend architecture of EKOA, including the technology stack, feature-based organization, routing, authentication, state management, design system, AI chat experience, knowledge management interface, workflow builder, accessibility, responsiveness, and testing strategy. The frontend is designed to provide a modern enterprise user experience while remaining maintainable and aligned with the platform's overall architecture.

Chapter 17: Backend Internal Architecture & Service Design

17.1 Introduction

The backend is responsible for coordinating business logic, AI orchestration, authentication, integrations, workflow execution, and data persistence.

The architecture follows Clean Architecture principles to achieve:

- High maintainability
- Low coupling
- High cohesion
- Testability
- Extensibility
- Framework independence
- The backend is implemented using Python, FastAPI, and asynchronous programming where appropriate.

17.2 Architectural Layers

The backend is divided into five primary layers:

```
Presentation Layer
        │
        ▼
Application Layer
        │
        ▼
Domain Layer
        │
        ▼
Infrastructure Layer
        │
        ▼
External Systems
```

Each layer depends only on abstractions provided by the layer beneath it.

17.3 Presentation Layer

The Presentation Layer exposes HTTP and WebSocket interfaces.

Responsibilities:

- Route handling
- Authentication
- Request validation
- Response serialization
- Streaming responses
- API documentation
- FastAPI routers are organized by feature:

```
routers/
├── auth.py
├── users.py
├── documents.py
├── chat.py
├── workflows.py
├── integrations.py
├── analytics.py
```

No business logic resides in the router layer.

17.4 Application Layer

The Application Layer orchestrates use cases.

Examples:

- UploadDocumentUseCase
- ExecuteWorkflowUseCase
- ChatWithKnowledgeUseCase
- CreateOrganizationUseCase
- InviteUserUseCase
- Responsibilities:
- Coordinate domain services.
- Manage transactions.
- Invoke repositories.
- Trigger background jobs.
- Publish domain events.

17.5 Domain Layer

The Domain Layer contains the core business rules.

Key elements:

- Entities

- Value Objects
- Domain Services
- Aggregates
- Domain Events
- Repository Interfaces
- Examples of entities:
- User
- Organization
- Workspace
- Document
- Conversation
- Workflow
- This layer has no dependency on FastAPI, SQLAlchemy, or external libraries.

17.6 Infrastructure Layer

The Infrastructure Layer provides concrete implementations.

Examples:

- PostgreSQL repositories
- Qdrant adapters
- Redis cache
- Object storage
- Email provider
- OAuth providers
- LangChain integrations
- MCP client
- External APIs
- Changing an implementation should not affect the Domain Layer.

17.7 Dependency Injection

FastAPI’s dependency system is used to inject services.

Typical dependencies include:

- Database session
- Current user
- Configuration
- AI service
- Repository implementations
- Centralized dependency management simplifies testing and promotes loose coupling.

17.8 Repository Pattern

Repositories abstract data persistence.

Example interfaces:

- UserRepository
- DocumentRepository
- WorkflowRepository
- ConversationRepository
- Responsibilities:
- CRUD operations
- Query methods
- Transaction support
- The Application Layer depends on repository interfaces rather than database-specific code.

17.9 Service Layer

Business services encapsulate reusable logic.

Examples:

- AuthenticationService
- DocumentService
- WorkflowService
- AIService
- NotificationService
- IntegrationService
- Services coordinate multiple repositories and external systems when necessary.

17.10 Domain Events

Significant business actions emit domain events.

Examples:

- UserRegistered
- DocumentUploaded
- WorkflowCompleted
- ConversationStarted
- IntegrationConnected
- Events enable decoupled communication between components.

17.11 Background Processing

Long-running tasks are delegated to worker services.

Examples:

- OCR
- Embedding generation
- Video transcription
- Notification delivery
- Scheduled workflows
- Jobs are queued using Redis and processed asynchronously.

17.12 Configuration Management

Configuration is centralized.

Categories include:

- Database
- Authentication
- AI providers
- Integrations
- Monitoring
- Feature flags
- Configuration is validated at startup using Pydantic settings.

17.13 Error Handling

The backend defines a consistent exception hierarchy.

Examples:

- ValidationError
- AuthenticationError
- AuthorizationError
- ResourceNotFoundError
- WorkflowExecutionError
- IntegrationError
- Exceptions are translated into standardized API responses.

17.14 Validation

Input validation occurs at multiple levels:

- API request validation (Pydantic)
- Domain rule validation
- Database constraints
- This layered approach prevents invalid state from entering the system.

17.15 Middleware

Core middleware includes:

- Request logging
- Correlation ID injection
- Authentication
- Rate limiting
- CORS
- Compression
- Performance metrics
- Middleware is applied consistently across all endpoints.

17.16 API Versioning

Routes are grouped by version.

Example:

/api/v1/ /api/v2/ Breaking changes require a new version, while backward-compatible enhancements remain within the current version.

17.17 Asynchronous Programming

FastAPI’s asynchronous capabilities are used for:

- External API calls
- Database operations (where supported)
- AI requests
- Streaming responses
- Background coordination
- CPU-intensive tasks are delegated to worker processes to avoid blocking the event loop.

17.18 Internal SDK

Shared functionality is exposed through internal SDKs.

Examples:

- AI SDK
- Integration SDK
- Notification SDK
- Storage SDK
- This reduces duplication across services.

17.19 Logging

Every service emits structured logs.

Fields include:

- Timestamp
- Service
- Correlation ID
- User ID
- Organization ID
- Severity
- Message
- Logs integrate with the centralized observability stack.

17.20 Caching Strategy

Redis is used to cache:

- Frequently accessed metadata
- User sessions
- AI model metadata
- Search results (where appropriate)
- Configuration
- Cache invalidation policies are defined per data type.

17.21 Event-Driven Communication

Internal events enable asynchronous workflows.

Examples:

```
DocumentUploaded
        │
        ├── GenerateEmbeddings
        ├── NotifyUser
        ├── UpdateSearchIndex
        └── WriteAuditLog
```

This reduces coupling between services and improves scalability.

17.22 Security Integration

Security is enforced throughout the backend.

Mechanisms include:

- Authorization decorators.
- Permission checks in services.
- Tenant isolation.
- Secret management.
- Secure logging.
- Audit event generation.
- Business logic never assumes authentication; it is always verified explicitly.

17.23 Testing Strategy

Backend testing covers:

- Unit tests for domain logic.
- Repository tests.
- API integration tests.
- Worker tests.
- AI service mocks.
- Contract tests.
- Each layer is tested independently to simplify debugging.

17.24 Extensibility

The architecture supports future enhancements such as:

- Additional AI providers.
- New integrations.
- Alternative databases.
- New workflow engines.
- Event bus adoption.
- Kubernetes deployment.
- These additions require minimal changes to existing code due to the use of abstractions.

Chapter Summary

This chapter defined the internal backend architecture of EKOA using Clean Architecture and Domain-Driven Design principles. By separating concerns into presentation, application, domain, and infrastructure layers, the backend remains modular, testable, and resilient to future changes while supporting AI orchestration, enterprise integrations, and scalable service design.

Chapter 18: Testing Strategy, Quality Assurance & AI Evaluation

18.1 Introduction

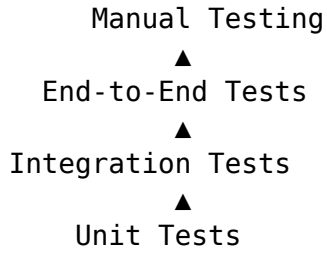
Testing within EKOA extends beyond traditional software verification to include AI model evaluation, retrieval quality assessment, workflow validation, and enterprise integration testing.

The testing strategy aims to ensure:

- Functional correctness
- Reliability
- Security
- Performance
- AI response quality
- Workflow stability
- Regression prevention
- Testing is integrated throughout the development lifecycle.

18.2 Testing Pyramid

The platform follows a layered testing strategy.



Most tests should be unit tests, with progressively fewer integration and end-to-end tests.

18.3 Unit Testing

Unit tests validate isolated components.

Coverage includes:

- Domain entities
- Services
- Utility functions
- Validation rules
- Prompt builders
- Repository interfaces
- Recommended tools:
- pytest
- pytest-asyncio
- unittest.mock
- Target coverage:
- ≥ 90% for critical business logic.

18.4 Integration Testing

Integration tests verify communication between components.

Examples:

- FastAPI ↔ PostgreSQL
- FastAPI ↔ Redis
- FastAPI ↔ Qdrant
- AI Service ↔ LangChain
- AI Service ↔ Ollama
- MCP Client ↔ MCP Server
- These tests ensure services interact correctly under realistic conditions.

18.5 End-to-End (E2E) Testing

E2E tests simulate complete user workflows.

Example scenarios:

- User registration and login.
- Document upload and indexing.
- AI chat with RAG.
- Workflow execution.
- Integration setup.
- Role-based access validation.
- Recommended tool:
- Playwright

18.6 API Contract Testing

API contracts ensure compatibility between frontend and backend.

Tests validate:

- Request schemas
- Response schemas
- Error responses
- Authentication behavior
- Version compatibility
- Contract testing reduces integration issues during deployment.

18.7 Performance Testing

Performance testing measures system responsiveness under load.

Metrics include:

- API latency
- Throughput
- Concurrent users
- Database response time
- Vector search latency
- AI response time
- Recommended tools:
- k6
- Locust

18.8 Load and Stress Testing

Load testing validates expected operational capacity.

Stress testing determines system behavior beyond expected limits.

Scenarios:

- Thousands of concurrent chat sessions.
- Large document ingestion.
- Massive workflow execution.
- High-frequency retrieval requests.
- Results inform scaling decisions.

18.9 Security Testing

Security validation includes:

- Authentication testing.
- Authorization testing.
- Rate limiting.
- Input validation.
- File upload validation.
- Dependency scanning.

- Penetration testing.
- Regular security assessments are integrated into the CI/CD pipeline.

18.10 AI Response Evaluation

AI-generated responses are evaluated against predefined criteria.

Evaluation dimensions:

- Relevance
- Accuracy
- Completeness
- Clarity
- Citation quality
- Safety
- Hallucination rate
- Both automated and human evaluations are performed.

18.11 RAG Evaluation

Retrieval-Augmented Generation quality is measured independently from the LLM.

Metrics include:

Metric	Description
Precision@k	Relevance of retrieved chunks
Recall@k	Coverage of relevant information
MRR	Mean Reciprocal Rank
nDCG	Ranking quality
Context Recall	Whether necessary context was retrieved

Evaluation datasets should contain representative enterprise queries and expected source documents.

18.12 Agent Evaluation

Each AI agent is evaluated individually.

Assessment criteria:

- Task completion rate
- Latency
- Tool selection accuracy
- Error recovery
- Coordination effectiveness
- Resource usage
- Agent metrics are tracked over time to identify regressions.

18.13 Workflow Validation

LangGraph workflows undergo automated validation.

Checks include:

- Correct node execution.
- Branching logic.
- Retry behavior.
- Human approval flow.
- State persistence.
- Failure recovery.
- Every workflow template has associated test cases.

18.14 Prompt Regression Testing

Prompt changes can unintentionally reduce response quality.

Regression testing compares:

- Previous prompt version.
- New prompt version.
- Evaluation considers:
- Accuracy
- Consistency
- Hallucination rate
- Formatting
- Token usage
- Prompt updates are approved only if quality is maintained or improved.

18.15 Benchmark Dataset

The platform maintains curated evaluation datasets.

Categories:

- HR
- Legal
- Finance
- Engineering
- Policies
- Technical documentation
- Customer support
- Each dataset includes:
- Query
- Expected sources
- Expected answer characteristics

18.16 Hallucination Detection

Responses are analyzed for unsupported claims.

Detection methods include:

- Citation verification.
- Retrieval consistency.
- LLM-as-a-judge evaluation.
- Rule-based validation.
- High-risk responses may require user warnings or human review.

18.17 Human Evaluation

Expert reviewers periodically assess AI outputs.

Evaluation criteria:

- Usefulness
- Correctness
- Readability
- Trustworthiness
- Compliance with organizational policies
- Human feedback informs future prompt and model improvements.

18.18 Continuous Quality Monitoring

Production systems continuously collect quality metrics.

Examples:

- User feedback ratings.
- AI response latency.
- Retrieval latency.
- Workflow success rate.
- Tool invocation failures.
- Model availability.
- Dashboards visualize quality trends over time.

18.19 Test Data Management

Test environments use sanitized or synthetic datasets.

Guidelines:

- No production secrets.
- No sensitive personal data.
- Representative enterprise documents.
- Version-controlled datasets.

- This supports reproducible testing while protecting privacy.

18.20 CI/CD Quality Gates

Every pull request must satisfy quality gates before merging.

Required checks:

- Unit tests.
- Integration tests.
- Linting.
- Type checking.
- Security scanning.
- AI regression tests.
- Coverage thresholds.
- Deployments are blocked if critical checks fail.

18.21 Chaos Engineering (Future)

To improve resilience, future versions may introduce controlled fault injection.

Examples:

- Database outage.
- Vector database latency.
- AI provider failure.
- Redis unavailability.
- Network interruptions.
- The objective is to verify graceful degradation and recovery.

18.22 Documentation Testing

Documentation is treated as part of the product.

Checks include:

- API documentation accuracy.
- Architecture diagrams.
- Configuration examples.
- Deployment guides.
- Automated validation helps prevent outdated documentation.

18.23 Acceptance Criteria

A feature is considered complete only if:

- Functional requirements are met.
- Tests pass.

- Security validation succeeds.
- Performance targets are satisfied.
- Documentation is updated.
- AI evaluation meets predefined thresholds.
- This definition of “Done” ensures consistent quality across the platform.

18.24 Continuous Improvement

Testing is an ongoing process.

Quality metrics, user feedback, and operational insights are reviewed regularly to:

- Improve prompts.
- Refine workflows.
- Optimize retrieval.
- Enhance agent collaboration.
- Reduce operational risk.
- Continuous learning is central to maintaining enterprise-grade AI performance.

Chapter Summary

This chapter established a comprehensive quality strategy for EKOA, covering traditional software testing, AI-specific evaluation, RAG assessment, agent validation, workflow verification, prompt regression testing, performance and security testing, and continuous quality monitoring. Together, these practices ensure that both the software and AI components remain reliable, measurable, and production-ready throughout the platform’s lifecycle.

Chapter 19: Observability, Monitoring & AI Operations (AIOps)

19.1 Introduction

Observability enables engineers, administrators, and AI operators to understand the internal state of EKOA through telemetry, metrics, logs, traces, and AI-specific analytics.

The platform extends traditional DevOps monitoring with AI Operations (AIOps), allowing organizations to monitor not only infrastructure but also AI behavior, workflow execution, retrieval quality, and operational costs.

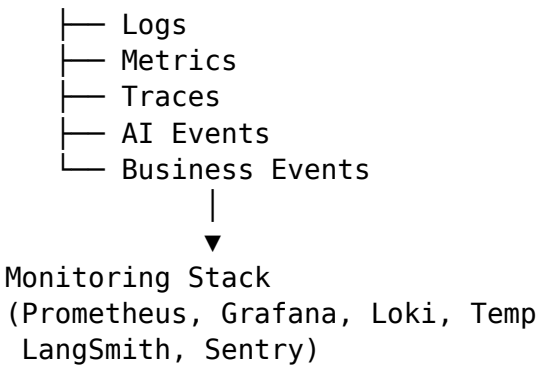
Objectives include:

- Detect failures quickly.
- Optimize AI performance.
- Monitor operational costs.
- Improve workflow efficiency.
- Support incident investigation.
- Enable capacity planning.

19.2 Observability Architecture

Users

```
   |
   ▼
Frontend
   |
   ▼
FastAPI API
   |
   ▼
AI Services
   |
   ▼
Telemetry Layer
   |
```



All services emit standardized telemetry.

19.3 Telemetry Categories

The platform collects five primary telemetry types:

- Infrastructure Metrics CPU usage
- Memory usage
- Disk utilization
- Network traffic
- Container health
- Application Metrics API requests
- Response latency
- Error rates
- Authentication failures
- Queue depth
- AI Metrics Token usage
- Prompt latency
- Model latency
- Hallucination indicators
- Retrieval quality
- Workflow Metrics Active workflows
- Completed workflows
- Failed workflows
- Retry counts
- Average execution time
- Business Metrics Daily active users
- Documents uploaded
- Conversations created
- Knowledge searches
- Workflow executions

19.4 Metrics Collection

Prometheus collects metrics from every service.

Examples:

- api_requests_total
- api_request_duration_seconds
- workflow_execution_total
- agent_execution_total
- retrieval_latency_seconds
- llm_request_duration_seconds Metrics are labeled with:
- Service
- Organization
- Environment
- Model
- Agent
- Workflow

19.5 Structured Logging

All services emit JSON logs.

Example fields:

- timestamp
- service
- organization_id
- user_id
- correlation_id
- workflow_id
- agent_name
- severity
- message Logs are centralized through Loki.

19.6 Distributed Tracing

Every request receives a trace ID.

Trace example:

```
Browser
   |
API
   |
LangGraph
   |
Retriever
   |
Qdrant
   |
LLM
```

Tempo visualizes end-to-end request execution.

19.7 AI Telemetry

Each LLM invocation records:

- Model name
- Provider
- Prompt version
- Input tokens
- Output tokens
- Total tokens
- Response latency
- Estimated cost
- Temperature
- Max tokens
- These metrics enable optimization across providers.

19.8 Prompt Analytics

Prompt execution history includes:

- Prompt identifier
- Version
- Agent
- Model
- Success rate
- Latency
- User feedback
- Evaluation score
- Prompt analytics help identify regressions after updates.

19.9 Agent Analytics

Each AI agent records:

- Invocations
- Average execution time
- Success rate
- Retry frequency
- Tool usage
- Failure categories
- Token consumption
- Administrators can compare agent performance over time.

19.10 Workflow Analytics

Workflow dashboards display:

- Running workflows
- Completion rates
- Average duration
- Approval wait times
- Retry statistics
- Failure reasons
- These insights support workflow optimization.

19.11 Retrieval Analytics

RAG performance metrics include:

- Retrieval latency
- Average retrieved chunks
- Reranking latency
- Citation coverage
- Precision@k
- Recall@k
- Cache hit rate
- Poor-performing knowledge collections can be identified quickly.

19.12 Token Usage Monitoring

Token consumption is tracked by:

- Organization
- User
- Workspace
- Agent
- Workflow
- Model
- Administrators can monitor trends and identify unusually expensive operations.

19.13 Cost Analytics

Estimated AI costs are calculated per request.

Breakdowns include:

- Daily cost
- Monthly cost
- Cost per organization
- Cost per workflow
- Cost per model
- Cost per user
- This enables informed decisions about model selection and optimization.

19.14 Model Health

Each configured model is continuously monitored.

Health indicators include:

- Availability
- Latency
- Error rate
- Timeout frequency
- Average response quality
- Throughput
- Fallback models may be selected automatically if thresholds are exceeded.

19.15 Dashboard Design

Grafana dashboards include:

- Infrastructure Dashboard CPU
- Memory
- Storage
- Containers
- API Dashboard Requests
- Errors
- Latency
- Active users
- AI Dashboard Tokens
- Costs
- Model usage
- Response times
- Workflow Dashboard Active workflows
- Success rates
- Retry counts
- Business Dashboard Documents
- Conversations
- Knowledge growth
- User activity

19.16 Alerting

Alerts are generated for:

- High error rates
- AI provider failures
- Excessive latency
- Queue backlogs
- Failed workflows
- Database connectivity issues

- Unusual token consumption
- Security events
- Critical alerts notify administrators immediately.

19.17 Incident Management

Incidents follow a structured lifecycle:

- Detection.
- Classification.
- Assignment.
- Investigation.
- Resolution.
- Root cause analysis.
- Preventive improvements.
- Operational runbooks guide responders through common scenarios.

19.18 Capacity Planning

Historical metrics support forecasting.

Examples:

- Storage growth
- Knowledge base expansion
- AI usage trends
- Workflow volume
- Concurrent users
- These forecasts inform infrastructure scaling decisions.

19.19 Service Level Objectives (SLOs)

Example objectives:

Metric	Target
API Availability	≥ 99.9%
AI Chat Response (P95)	< 5 seconds
Workflow Success Rate	≥ 99%
Authentication Success	≥ 99.9%
Document Indexing	< 2 minutes (standard-sized documents)

Service Level Indicators (SLIs) are continuously measured against these objectives.

19.20 Error Budgets

Error budgets define acceptable failure levels.

Examples:

- Monthly API downtime allowance.
- AI provider failure tolerance.
- Workflow retry thresholds.
- Exceeding an error budget triggers engineering review before new feature development continues.

19.21 Operational Reports

Administrators can generate periodic reports.

Examples:

- Weekly AI usage.
- Monthly infrastructure utilization.
- Retrieval quality trends.
- Cost optimization opportunities.
- Security events.
- Reports support governance and executive oversight.

19.22 Operational KPIs

Representative KPIs include:

- Mean Time to Detect (MTTD)
- Mean Time to Resolve (MTTR)
- Average AI response latency
- Cost per conversation
- Workflow completion rate
- Knowledge ingestion volume
- User satisfaction score
- KPIs are reviewed regularly to guide continuous improvement.

19.23 Continuous Optimization

Operational insights are used to:

- Tune prompts.
- Improve retrieval strategies.
- Adjust model routing.
- Optimize workflow design.
- Scale infrastructure.
- Reduce costs.

- Optimization is an ongoing process rather than a one-time activity.

19.24 Future Enhancements

Potential future capabilities include:

- Predictive failure detection using machine learning.
- Automatic anomaly detection.
- AI-assisted incident triage.
- Autonomous workflow optimization.
- Intelligent infrastructure scaling recommendations.
- These features align with the long-term evolution of AIOps.

Chapter Summary

This chapter defined the observability and operational management strategy for EKOA. By combining infrastructure monitoring, application telemetry, AI analytics, workflow metrics, cost tracking, distributed tracing, and operational dashboards, the platform provides comprehensive visibility into both technical health and AI behavior. This observability foundation enables proactive maintenance, informed optimization, and reliable enterprise operations.

Chapter 20: Enterprise Integrations & Connector Framework

20.1 Introduction

Enterprise organizations rely on numerous software platforms to manage projects, documents, communication, development, customer relationships, and business operations.

EKOA integrates with these systems through a standardized Connector Framework that provides:

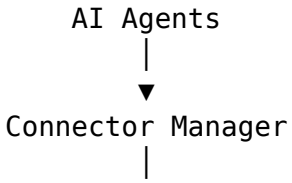
- Secure authentication
- Unified execution
- Data synchronization
- Event handling
- Monitoring
- Permission enforcement
- The framework supports both built-in and custom connectors.

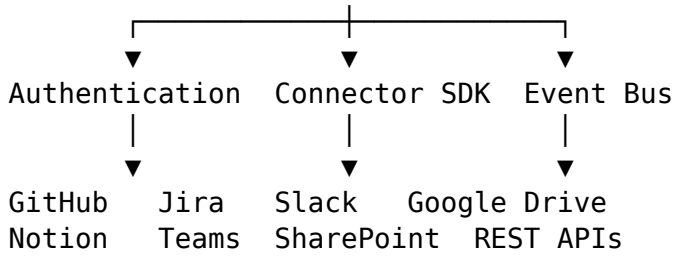
20.2 Integration Objectives

The connector architecture is designed to:

- Eliminate custom integration logic.
- Standardize authentication.
- Simplify maintenance.
- Support enterprise scalability.
- Enable plugin-based extensibility.
- Preserve security and auditability.

20.3 Connector Architecture





The Connector Manager acts as the central coordination layer.

20.4 Connector Lifecycle

Each connector follows a consistent lifecycle:

- Installation
- Configuration
- Authentication
- Permission validation
- Synchronization
- Runtime execution
- Monitoring
- Update
- Disable
- Uninstallation
- This lifecycle simplifies administration and troubleshooting.

20.5 Connector Interface

Every connector implements a common interface.

Core operations include:

- Connect
- Disconnect
- Authenticate
- Refresh credentials
- Validate permissions
- List resources
- Execute actions
- Handle events
- Report health
- This abstraction allows the platform to interact with connectors uniformly.

20.6 Authentication Methods

Supported authentication mechanisms:

- OAuth 2.0

- API Keys
- Personal Access Tokens
- Service Accounts
- Client Credentials
- JWT (internal services)
- Credentials are encrypted and managed securely.

20.7 GitHub Integration

Capabilities include:

- Repository browsing
- Code search
- Pull request summaries
- Issue management
- Commit history
- Workflow status
- Release information
- Future enhancements:
- Automated code review
- Repository knowledge indexing
- CI/CD insights

20.8 Jira Integration

Supported features:

- Issue creation
- Issue updates
- Sprint information
- Project metadata
- Workflow transitions
- Comment management
- AI agents can generate Jira issues from meeting notes, audit findings, or workflow outputs.

20.9 Slack Integration

Features:

- Channel messaging
- Direct messages
- Workflow notifications
- AI summaries
- Interactive approvals
- Slack notifications respect organization-specific preferences.

20.10 Microsoft Teams

Capabilities include:

- Team notifications
- Meeting summaries
- Approval requests
- AI assistant integration
- Workflow alerts
- Teams integration supports enterprise collaboration scenarios.

20.11 Google Workspace

Supported services:

- Google Drive
- Google Docs
- Google Sheets
- Google Calendar
- Gmail
- Example capabilities:
- Document ingestion
- Calendar scheduling
- Email drafting
- Spreadsheet analysis

20.12 Microsoft 365

Supported services:

- OneDrive
- Outlook
- SharePoint
- Excel
- Word
- PowerPoint
- Enterprise document repositories can be indexed while respecting existing access controls.

20.13 Notion Integration

Capabilities:

- Page synchronization
- Knowledge indexing
- Database retrieval
- Workspace search

- Notion becomes another searchable knowledge source for RAG.

20.14 Confluence Integration

Features:

- Space synchronization
- Page indexing
- Version tracking
- Metadata extraction
- Confluence content integrates seamlessly into the enterprise knowledge base.

20.15 REST API Connector

Organizations can connect custom systems using REST APIs.

Supported features:

- GET
- POST
- PUT
- PATCH
- DELETE
- Configuration includes:
- Authentication
- Headers
- Timeouts
- Retry policies
- This connector enables integration with proprietary enterprise systems.

20.16 Webhook Framework

EKOA supports inbound and outbound webhooks.

Examples:

- Inbound:
- New issue created
- Document updated
- Employee onboarded
- Outbound:
- Workflow completed
- AI report generated
- Security alert
- Webhook signatures verify authenticity.

20.17 Event-Driven Integrations

Connectors can subscribe to platform events.

Examples:

- DocumentUploaded
- ↓
- SharePoint Sync
- ↓
- Notification
- ↓
- Audit Log Event-driven execution reduces polling and improves responsiveness.

20.18 Synchronization Strategies

Supported synchronization modes:

- Manual
- Scheduled
- Event-driven
- Incremental
- Full synchronization
- Administrators configure synchronization frequency per connector.

20.19 Connector Health Monitoring

Health metrics include:

- Connectivity
- Authentication status
- Synchronization latency
- Error rate
- Last successful sync
- API quota usage
- Health information is surfaced through the monitoring dashboard.

20.20 Permission Model

Connector permissions follow the principle of least privilege.

Examples:

- Read repositories
- Create issues
- Send messages
- Read documents
- Upload files

- Users grant only the permissions required for the intended functionality.

20.21 Audit & Compliance

All connector activities are logged.

Recorded information includes:

- User
- Connector
- Action
- Timestamp
- Resource
- Outcome
- This supports compliance and forensic investigations.

20.22 Connector SDK

Developers can build custom connectors using the Connector SDK.

SDK capabilities include:

- Authentication helpers
- Configuration management
- Event subscriptions
- Logging
- Health reporting
- Retry handling
- Error mapping
- This reduces the effort required to support additional enterprise systems.

20.23 Failure Handling

Connector failures are managed gracefully.

Strategies include:

- Automatic retries
- Exponential backoff
- Circuit breaker patterns
- User notifications
- Detailed diagnostics
- Critical failures trigger operational alerts.

20.24 Future Enhancements

Potential future connectors include:

- Salesforce
- SAP
- ServiceNow
- Zendesk
- HubSpot
- AWS
- Azure
- Kubernetes
- Snowflake
- Databricks
- The Connector Framework is designed to accommodate new integrations without architectural changes.

Chapter Summary

This chapter defined the Enterprise Connector Framework for EKOA. By standardizing connector lifecycles, authentication, synchronization, monitoring, and permissions, the platform enables secure and extensible integration with a broad ecosystem of enterprise applications. The plugin-based architecture ensures that new integrations can be added consistently while maintaining governance, observability, and operational reliability.

Chapter 21: Performance Engineering, Scalability & Future Roadmap

21.1 Introduction

Performance engineering ensures that EKOA remains responsive, reliable, and cost-effective as workloads increase. Scalability planning allows the platform to support larger organizations, additional users, more AI models, and growing knowledge repositories without major architectural redesign.

This chapter defines optimization strategies, scaling patterns, and a long-term roadmap for the platform's evolution.

21.2 Performance Objectives

Representative performance targets include:

Metric	Target
Initial page load	< 2 seconds
API response (P95)	< 300 ms (non-AI)
AI response start	< 2 seconds (streaming)
AI response completion (P95)	< 5 seconds (standard prompts)
Document indexing	< 2 minutes (standard-sized files)
Vector search latency	< 200 ms
Authentication	< 500 ms

Performance budgets are reviewed periodically as workloads evolve.

21.3 Scalability Principles

The architecture follows these principles:

- Stateless application services.
- Horizontal scaling for compute-heavy services.
- Independent scaling per service.
- Asynchronous processing for long-running tasks.

- Efficient caching.
- Database optimization.
- AI model abstraction.

21.4 Horizontal Scaling

The following services are designed for horizontal scaling:

- FastAPI API
- AI Service
- Worker Service
- MCP Server
- Notification Service
- Load balancers distribute requests across instances to improve throughput and resilience.

21.5 Vertical Scaling

Certain components may initially benefit more from vertical scaling:

- PostgreSQL
- Qdrant
- Redis
- As usage grows, clustering and replication strategies can be introduced where supported.

21.6 Multi-Level Caching

Caching reduces latency and infrastructure costs.

Browser Cache

- Static assets
- Images
- Fonts
- CDN Cache Public assets
- Documentation
- Application Cache (Redis) User sessions
- Configuration
- Frequently accessed metadata
- Workflow definitions
- AI Cache Embeddings
- Prompt templates
- Model metadata
- Reusable retrieval results (where appropriate)
- Cache invalidation policies are defined for each cache layer.

21.7 Database Optimization

Optimization techniques include:

- Proper indexing.
- Query optimization.
- Connection pooling.
- Pagination.
- Partitioning (future).
- Read replicas (future).
- Database performance is monitored continuously.

21.8 Vector Search Optimization

Strategies include:

- Efficient metadata filtering.
- HNSW index tuning (Qdrant).
- Hybrid retrieval.
- Embedding normalization.
- Incremental indexing.
- Reranking only when necessary.
- These techniques balance accuracy and latency.

21.9 AI Model Routing

The AI Router selects the most appropriate model for each request.

Selection criteria may include:

- Task type
- Latency requirements
- Context size
- Cost
- Availability
- Organization preferences
- Example routing:

Task	Preferred Model
General chat	DeepSeek
Long-context summarization	Long-context model (future)
Coding assistance	Code-specialized model
Lightweight tasks	Ollama-hosted local model

Routing rules are configurable and versioned.

21.10 Cost Optimization

Cost management strategies include:

- Local inference where practical.
- Response streaming.
- Prompt optimization.
- Context pruning.
- Model routing.
- Retrieval caching.
- Token budgeting.
- Administrators can define usage policies and budgets.

21.11 Knowledge Base Growth

As the knowledge base expands:

- Incremental indexing is preferred.
- Collections are partitioned logically.
- Metadata filters reduce search scope.
- Archival policies manage inactive content.
- This prevents retrieval performance from degrading over time.

21.12 Workflow Optimization

Workflow performance is improved through:

- Parallel execution.
- Conditional branching.
- Reusable workflow templates.
- Checkpointing.
- Efficient retry policies.
- Long-running workflows avoid blocking interactive requests.

21.13 Connector Optimization

Connector efficiency is improved through:

- Incremental synchronization.
- Event-driven updates.
- Connection pooling.
- Intelligent retry logic.
- Rate limit awareness.
- These practices reduce unnecessary external API traffic.

21.14 Sustainability Considerations

The platform aims to use compute resources responsibly.

Examples:

- Shut down idle worker processes where appropriate.
- Prefer efficient models for simple tasks.
- Cache reusable computations.
- Minimize redundant indexing.
- These practices reduce both operational cost and environmental impact.

21.15 Technical Debt Management

Technical debt is tracked intentionally.

Categories include:

- Architecture
- Code quality
- Documentation
- Testing
- Infrastructure
- AI prompts
- Workflows
- Debt items are prioritized based on risk and business value.

21.16 Product Roadmap

Phase 1 – Foundation Authentication

Knowledge ingestion

- RAG
- AI chat
- Basic workflows
- Phase 2 – Enterprise Features Multi-agent orchestration
- MCP support
- Connectors
- Analytics
- Administration
- Phase 3 – Advanced Automation Visual workflow builder
- Advanced approvals
- AI recommendations
- Expanded integrations
- Phase 4 – Intelligent Platform Autonomous workflow optimization
- Predictive analytics
- AI-assisted administration
- Advanced knowledge graph capabilities

21.17 Research Directions

Potential research topics include:

- Adaptive retrieval strategies.
- Agent collaboration optimization.
- Continual learning.
- Multimodal reasoning.
- Graph-enhanced RAG.
- Retrieval quality prediction.
- Human-AI collaboration models.
- These areas can guide future academic or product work.

21.18 Emerging AI Capabilities

The architecture is designed to support future developments such as:

- Larger context windows.
- Improved reasoning models.
- Native multimodal models.
- Voice-first interfaces.
- On-device inference.
- Federated AI deployments.
- Abstraction layers allow these capabilities to be adopted incrementally.

21.19 Long-Term Architecture Evolution

Potential future architectural enhancements include:

- Kubernetes deployment.
- Service mesh.
- Event streaming platform (e.g., Kafka).
- Knowledge graph integration.
- Federated search across organizations (subject to trust policies).
- Multi-region deployments.
- Edge AI inference.
- These enhancements can be introduced without fundamental redesign.

21.20 Success Metrics

Platform success can be evaluated using:

- Technical Availability
- Latency
- Error rates
- Deployment frequency
- Mean Time to Recovery (MTTR)

- AI Retrieval precision
- Citation coverage
- User satisfaction
- Hallucination rate
- Workflow completion rate
- Business User adoption
- Active organizations
- Knowledge growth
- Connector usage
- Operational cost per user
- These metrics support continuous improvement.

21.21 Architectural Decision Records (ADRs)

Significant architectural decisions should be documented.

Each ADR includes:

- Context
- Decision
- Alternatives considered
- Consequences
- Status
- Examples:
- Why FastAPI was selected.
- Why Qdrant was chosen over alternatives.
- Why LangGraph was adopted.
- Why hybrid RBAC/ABAC was implemented.
- ADRs preserve architectural knowledge as the platform evolves.

21.22 Documentation Strategy

Architecture documentation should evolve alongside the codebase.

Key artifacts include:

- ESAS document (this specification)
- API documentation
- Deployment guides
- Runbooks
- ADRs
- Sequence diagrams
- ER diagrams
- User guides
- Developer onboarding guides
- Documentation is version-controlled and reviewed as part of the development process.

21.23 Vision Statement

EKOA is envisioned as more than an AI assistant. It is an Enterprise Knowledge Operations Platform that unifies organizational knowledge, intelligent automation, AI agents, and enterprise integrations into a secure, extensible, and observable ecosystem.

Its modular architecture enables organizations to adopt new AI models, integrations, and workflows while preserving governance, scalability, and maintainability.

21.24 Final Summary

This chapter concluded the Enterprise Software Architecture Specification by defining the platform's performance objectives, scalability strategy, optimization techniques, long-term roadmap, and architectural evolution. Together with the preceding chapters, it provides a comprehensive blueprint for designing, implementing, operating, and evolving EKOA as an enterprise-grade AI platform.

Appendix A: Planned Future Documentation (Not Yet Drafted)

The core specification in this document (Chapters 2–21) forms **Volume I — Enterprise Software Architecture**. During planning, this was scoped as the first of a larger four-volume documentation set. The remaining volumes and appendices were outlined but not written, and are listed here as a roadmap for extending this specification into a complete implementation blueprint.

A.1 Additional Planned Volumes

- **Volume II — Developer Implementation Guide**
- **Volume III — API & AI Agent Reference**
- **Volume IV — Deployment, Operations & DevOps Guide**

A.2 Planned Reference Appendices

- **Appendix B** — Complete Folder Structure (Frontend, Backend, AI Services)
- **Appendix C** — Database Schema (All PostgreSQL Tables)
- **Appendix D** — Complete ER Diagram
- **Appendix E** — Qdrant Collection Design
- **Appendix F** — Redis Key Design
- **Appendix G** — API Reference (Every REST Endpoint)
- **Appendix H** — WebSocket Event Reference
- **Appendix I** — MCP Tool Specifications
- **Appendix J** — LangGraph Workflow Definitions
- **Appendix K** — Prompt Library Specification
- **Appendix L** — Agent Specification (Inputs, Outputs, States)
- **Appendix M** — Sequence Diagrams (50–100 diagrams)
- **Appendix N** — Deployment Diagrams
- **Appendix O** — Docker Compose & Infrastructure Files
- **Appendix P** — GitHub Actions CI/CD Pipelines
- **Appendix Q** — Security Threat Matrix (STRIDE)
- **Appendix R** — Testing Matrix & Coverage Plan
- **Appendix S** — Coding Standards & Conventions
- **Appendix T** — Glossary

- **Appendix U** — Architectural Decision Records (ADRs)

Completing these would turn this specification into a full, developer-ready implementation package covering everything from initial architecture through production deployment.